

Taylor & Francis
Taylor & Francis Group

Journal of Computational and Graphical Statistics

ISSN: 1061-8600 (Print) 1537-2715 (Online) Journal homepage: www.tandfonline.com/journals/ucgs20

Sparsification Subsampling for Partial Least Squares Regression

Junyi Lin, Mengyu Li, Cheng Meng & Yongdao Zhou

To cite this article: Junyi Lin, Mengyu Li, Cheng Meng & Yongdao Zhou (31 Jul 2026): Sparsification Subsampling for Partial Least Squares Regression, Journal of Computational and Graphical Statistics, DOI: 10.1080/10618600.2026.2686429

To link to this article: https://doi.org/10.1080/10618600.2026.2686429

View supplementary material

Published online: 31 Jul 2026.

Submit your article to this journal

Article views: 119

View related articles

View Crossmark data

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=ucgs20

JOURNAL OF COMPUTATIONAL AND GRAPHICAL STATISTICS
https://doi.org/10.1080/10618600.2026.2686429

Taylor & Francis
by informa

Check for updates

Sparsification Subsampling for Partial Least Squares Regression

Junyi Lin[a], Mengyu Li[b,#], Cheng Meng[c], and Yongdao Zhou[d]

[a]Institute of Statistics and Big Data, Renmin University of China, Beijing, China; [b]Department of Statistics and Data Science, Tsinghua University, Beijing, China; [c]Center for Applied Statistics, Institute of Statistics and Big Data, Renmin University of China, Beijing, China; [d]NITFID, School of Statistics and Data Science, Nankai University, Tianjin, China

ABSTRACT

Partial Least Squares (PLS) regression is a powerful tool for dimensionality reduction, multi-response variable regression, and multi-classification. However, its application to large-scale datasets is often hampered by significant computational burdens. Although variants of PLS algorithms have been proposed to address this challenge, an efficient algorithm that significantly reduces the time complexity remains underexplored. In this paper, we propose a novel PLS regression method based on sparsification subsampling to address this gap. We first construct a sparse sketch matrix using element-wise subsampling to obtain an unbiased approximation of the covariance matrix. Based on this, we propose a core-elements estimator that effectively incorporates the subsampled sparse matrix into the PLS regression. The resulting algorithm employs a kernel-based optimization framework to improve computational efficiency. Theoretically, we demonstrate that under specific probabilities, the approximate matrix obtained by sparsification subsampling achieves minimal variance for any given projection matrix. Finally, a series of Monte Carlo simulations and real-world data analyses validate the superior performance of our approach in terms of estimation accuracy, prediction power, and computational efficiency. Compared to row-wise sampling methods and even full-sample PLS, the proposed algorithm exhibits significant advantages. Supplementary materials for this article are available online.

ARTICLE HISTORY
Received March 2025
Accepted May 2026

KEYWORDS
Approximate matrix multiplication; Element-wise sampling; Large-scale data; Matrix sparsification; Multi-response regression

1. Introduction

Partial Least Squares (PLS) regression (Wold 1966; Höskuldsson 1988; Cook and Forzani 2024) is a multivariate statistical method well-suited for analyzing high-dimensional data, particularly in the presence of multiple response variables. Unlike classical linear regression, PLS regression assumes that the observed data can be represented by low-dimensional latent variables, which are iteratively identified as linear combinations of predictors that are most strongly correlated with the responses. By constructing these latent variables over a limited number of iterations, PLS regression achieves dimensionality reduction, effectively mitigating the issue of multicollinearity (Wold et al. 1984) often encountered in high-dimensional data.

Although PLS was originally designed for high-dimensional data with relatively small sample sizes, its computational efficiency has become increasingly important in the era of big data. Given a sample with n observations and p features, early implementations employed the computationally intensive power iteration method (Wold 1966), and subsequent improvements have brought the time complexity down to $\mathcal{O}(np^2)$ (De Jong 1993; Lindgren, Geladi, and Wold 1993; Dayal and MacGregor 1997), which is on the same order as ordinary least squares regression. Nevertheless, when the number of observations n becomes very large, the sheer volume of data can still pose a significant computational challenge. Various methods have been proposed to address this issue, including online PLS for streaming data (Zeng and Li 2014; Arora, Mianjy, and Marinov 2016; Stott et al. 2017; Jordao et al. 2021), acceleration using tree-based discriminant structures (Schwartz, Guo, and Davis 2010), semantic compression (Tabei et al. 2016), and parallelization through matrix partitioning techniques (Lafaye de Micheaux, Liquet, and Sutton 2019). Despite these advancements, many existing methods lack rigorous time complexity analyses or are only effective for specialized data structures.

Subsampling algorithms have emerged as a powerful tool for reducing the computational costs in large-scale statistical problems, including regression. These methods can be broadly categorized into probabilistic and deterministic approaches. Probabilistic methods often employ leverage scores (Drineas, Mahoney, and Muthukrishnan 2006; Ma, Mahoney, and Yu 2014; Ma and Sun 2015), influence functions (Ting and Brochu 2018), or various optimality criteria (Ma et al. 2022) to design sampling probabilities. Deterministic approaches include Information-Based Optimal Subsampling Selection (IBOSS) (Wang, Yang, and Stufken 2019; Yu, Liu, and Wang 2023), model-free optimal subsampling based on space-filling designs (Meng et al. 2021; Yi and Zhou 2023), optimal approximate orthogonal arrays (Wang et al. 2021), predictive inference-based subsampling (Wu et al. 2024), and uniform design-based methods (Zhang et al. 2024a; Zhou et al. 2024). The choice between probabilistic and deterministic approaches often

CONTACT Cheng Meng *chengmeng@ruc.edu.cn* Center for Applied Statistics, Institute of Statistics and Big Data, Renmin University of China, Beijing, China; Yongdao Zhou *ydzhou@nankai.edu.cn* NITFID, School of Statistics and Data Science, Nankai University, Tianjin, China.
#Joint first author.

Supplementary materials for this article are available online. Please go to. https:// www.tandfonline.com/r/JCGS.

© 2026 American Statistical Association and Institute of Mathematical Statistics

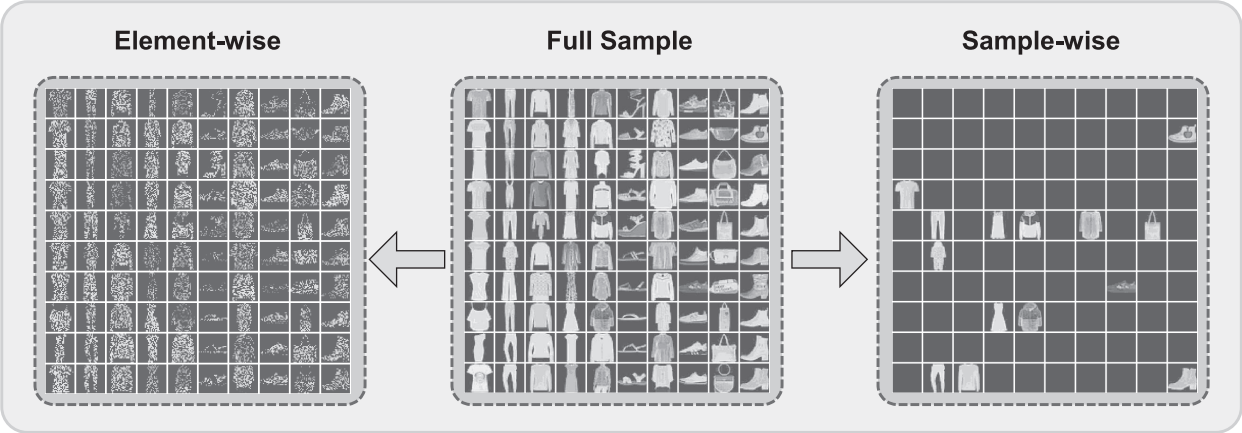


Figure 1. Illustration of element-wise subsampling versus sample-wise subsampling on the Fashion-MNIST dataset (Xiao, Rasul, and Vollgraf 2017). Notably, the element-wise method effectively preserves the key features of each image, whereas row sample-wise method may lead to the complete loss of information for unsampled images.

depends on specific model characteristics and parameters. Beyond improving computational efficiency, effective subsampling can also address challenges related to measurement constraints (Wang, Yu, and Singh 2017; Meng et al. 2021) and privacy protection (Wang, Balle, and Kasiviswanathan 2019).

Subsampling algorithms have been widely adopted in various statistical learning tasks, including least squares regression (Drineas, Mahoney, and Muthukrishnan 2006; Ma, Mahoney, and Yu 2014; Ma and Sun 2015; Meng et al. 2017; Ting and Brochu 2018; Wang, Yang, and Stufken 2019, Wang et al. 2021; Ma et al. 2022), generalized linear models (Wang, Zhu, and Ma 2018; Ai et al. 2021b; Yu et al. 2022, Yu, Wang, and Ai 2025), nonparametric regression (Ma, Huang, and Zhang 2015; Meng et al. 2020, 2022; Zhang et al. 2024b), quantile regression (Ai et al. 2021a; Wang and Ma 2021), model-free methods (Meng et al. 2021; Yi and Zhou 2023; Zhang et al. 2024a; Zhou et al. 2024), time series analysis (Xie et al. 2019, Xie, Bai, and Ma 2023), machine learning (Feng and Zhou 2024; Han et al. 2025; Huang et al. 2026). We also refer readers to Li and Meng (2020) and Yu, Ai, and Ye (2024) for a comprehensive overview. In the context of PLS regression, Xie, Feng, and Chen (2022) and Huang et al. (2024) proposed subsampling methods based on influence functions (Ting and Brochu 2018) and minimum covariance determinant to remove outliers and improve robustness. However, research on optimizing or improving the computational efficiency of the PLS algorithm from a time complexity perspective remains limited. This gap motivates us to explore new subsampling methods that reduce the computational burden of PLS while preserving its accuracy.

Beyond row sampling, another approach involves sampling individual elements of the observation matrix. As illustrated in Figure 1, rather than discarding entire rows, sparsification subsampling preserves the most crucial information within each sample. Moreover, Li et al. (2024) demonstrated that element-wise sampling achieves acceleration similar to row-wise sampling when extracting an equal number of elements. Regardless of the specific sampling strategy, the primary concern is the variance properties of the resulting parameter estimates. In PLS regression, the computation and updating of the kernel matrix are central to the algorithm (Lindgren, Geladi, and Wold 1993; Dayal and MacGregor 1997). This motivates us to seek a kernel estimation method that is both faster and exhibits lower variance. Notably, fast algorithms employing sparsification have been successfully applied in diverse fields such as low rank matrix approximations (Achlioptas and Mcsherry 2007; Xue et al. 2026), principal component analysis (Kundu, Drineas, and Magdon-Ismail 2017), linear regression (Li et al. 2024), nonparametric additive models (Li, Zhang, and Meng 2024), and optimal transport (Li et al. 2023a, 2023b; Hu et al. 2024; Ouyang et al. 2026). Inspired by the Core-Elements approach (Li et al. 2024), we incorporate a sparse matrix as an auxiliary variable into the original PLS algorithm and propose a novel kernel algorithm based on sparsification to reduce the computational complexity of PLS regression.

1.1. Our Contributions Are Summarized as Follows

First, we develop a new element-wise sampling method to approximate the covariance matrix, demonstrating its minimal variance under specific sampling probabilities. Moreover, this minimal variance property holds for any projection of the covariance matrix, offering a theoretical advantage over existing methods.

Second, we propose a Sparsification Subsampling for Partial Least Squares (Spar-PLS) algorithm that uses the sparse matrix obtained through our sampling method. We further enhance this algorithm by incorporating established kernel methods proposed by Lindgren, Geladi, and Wold (1993) and Dayal and MacGregor (1997).

Finally, through extensive Monte Carlo simulations, we evaluate the performance of our algorithm in terms of parameter estimation, prediction accuracy, and computational time. Our results show that the Spar-PLS method achieves comparable or superior estimation and prediction performance while reducing the running time by 80%-90%. Furthermore, we evaluate the performance of our algorithm on a wave energy converter dataset, where it consistently outperforms row sampling methods across various scenarios.

The remainder of this paper is organized as follows. Section 2 provides background on the PLS regression algorithms, matrix multiplication approximation algorithms, and element-wise sampling methods for least squares regression. Section 3

details the optimal sampling probability settings and introduces our proposed Spar-PLS method. Sections 4 and 5 present the results of numerical simulations and empirical data analyses, respectively, to evaluate the effectiveness and efficiency of our algorithm. Finally, Section 6 concludes the paper by summarizing our findings, discussing limitations, and suggesting directions for future research. Supplementary materials include notation, technical proofs, additional implementation details, and extended numerical experiments. The implementation code for the proposed method is available at the following link: https://github.com/junyilin559/Spar-PLS.

2. Background

We introduce some notation and definitions. The Euclidean norm (or ℓ_2 norm) of a vector $\boldsymbol{x}$ is defined as $\|\boldsymbol{x}\| = (\sum_i \boldsymbol{x}_i^2)^{1/2}$ (or $\|\boldsymbol{x}\|_2$). For a matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$, let $\boldsymbol{x}_i$ $(i = 1, \ldots, p)$ denote the i-th column of $\mathbf{X}$ as a column vector, and let $\boldsymbol{x}_{(j)}$ $(j = 1, \ldots, n)$ denote the j-th row of $\mathbf{X}$ as a column vector, its spectral norm (i.e., the largest singular value) and the Frobenius norm are denoted as $\|\mathbf{X}\|_2$ and $\|\mathbf{X}\|_F$, respectively.

2.1. Partial Least Squares

Partial Least Squares (PLS) regression is primarily employed to analyze data with multiple response variables. Consider a set of independently and identically distributed data $\{(\boldsymbol{x}_{(j)}, \boldsymbol{y}_{(j)})\}_{j=1}^n \subseteq \mathbb{R}^p \times \mathbb{R}^l$, where $\{\boldsymbol{x}_{(j)}\}_{j=1}^n \subset \mathbb{R}^p$ represent the predictor variables and $\{\boldsymbol{y}_{(j)}\}_{j=1}^n \subseteq \mathbb{R}^l$ represent the response variables. We denote the predictor matrix as $\mathbf{X} = (\boldsymbol{x}_{(1)}, \ldots, \boldsymbol{x}_{(n)})^\top \in \mathbb{R}^{n\times p}$ and the response matrix as $\mathbf{Y} = (\boldsymbol{y}_{(1)}, \ldots, \boldsymbol{y}_{(n)})^\top \in \mathbb{R}^{n\times l}$. PLS regression is particularly suitable for datasets where the predictors exhibit multicollinearity, making $\mathbf{X}$ be of low rank.

To identify the low-rank structure of $\mathbf{X}$ for predicting the response variables, PLS introduces a latent variable matrix $\mathbf{T} = (\boldsymbol{t}_1, \ldots, \boldsymbol{t}_m) \in \mathbb{R}^{n\times m} (m \leq p)$ and considers the following regression model:

$$\begin{cases} \mathbf{X} = \mathbf{T}\mathbf{P}^\top + \mathbf{E}, \\ \mathbf{Y} = \mathbf{T}\mathbf{Q}^\top + \mathbf{F}, \end{cases} \tag{1}$$

where $\mathbf{P} \in \mathbb{R}^{p\times m}$ and $\mathbf{Q} \in \mathbb{R}^{l\times m}$ are the loading matrices, and $\mathbf{E} \in \mathbb{R}^{n\times p}$ and $\mathbf{F} \in \mathbb{R}^{n\times l}$ are the residual matrices. To estimate $\mathbf{T}$ and the associated loading matrices, we initially assume its first component as $\boldsymbol{t}_1 = \mathbf{X}\boldsymbol{v}_1$ and $\boldsymbol{t}_1 = \mathbf{Y}\boldsymbol{w}_1$. However, if $\mathbf{Y}$ is not an exact linear combination of $\mathbf{X}$, these two equations cannot be simultaneously satisfied. Consequently, we seek an approximate $\boldsymbol{t}_1$ that optimally satisfies these conditions by maximizing the covariance between the latent components derived from $\mathbf{X}$ and $\mathbf{Y}$. For clarity in the subsequent steps, we define $\mathbf{X}_1 = \mathbf{X}$ and $\mathbf{Y}_1 = \mathbf{Y}$.

Considering the two expressions for the first latent variable, $\boldsymbol{t}_1$, namely $\mathbf{X}_1\boldsymbol{v}_1$ and $\mathbf{Y}_1\boldsymbol{w}_1$, the objective is to identify $\boldsymbol{v}_1$ and $\boldsymbol{w}_1$ that maximize the inner product of these two vectors, that is,

$$\begin{aligned} \boldsymbol{v}_1, \boldsymbol{w}_1 = & \underset{\boldsymbol{v}_1\in\mathbb{R}^p, \boldsymbol{w}_1\in\mathbb{R}^l}{\arg\max} \left\{ (\mathbf{X}_1\boldsymbol{v}_1)^\top \mathbf{Y}_1\boldsymbol{w}_1 \right\} \\ \text{s.t. } & \boldsymbol{v}_1^\top\boldsymbol{v}_1 = 1, \ \boldsymbol{w}_1^\top\boldsymbol{w}_1 = 1. \end{aligned} \tag{2}$$

Setting $\boldsymbol{t}_1 = \mathbf{X}_1\boldsymbol{v}_1$, we can then decompose $\mathbf{X}_1$ and $\mathbf{Y}_1$ as follows:

$$\begin{cases} \mathbf{X}_1 = \boldsymbol{t}_1\boldsymbol{p}_1^\top + \mathbf{X}_2, \\ \mathbf{Y}_1 = \boldsymbol{t}_1\boldsymbol{q}_1^\top + \mathbf{Y}_2, \end{cases} \tag{3}$$

where the loading vectors $\boldsymbol{p}_1$ and $\boldsymbol{q}_1$ are estimated via least squares:

$$\boldsymbol{p}_1^\top = (\boldsymbol{t}_1^\top\boldsymbol{t}_1)^{-1}\boldsymbol{t}_1^\top\mathbf{X}_1, \quad \boldsymbol{q}_1^\top = (\boldsymbol{t}_1^\top\boldsymbol{t}_1)^{-1}\boldsymbol{t}_1^\top\mathbf{Y}_1. \tag{4}$$

For the residual matrices in (3), defined as $\mathbf{X}_2 = \mathbf{X}_1 - \boldsymbol{t}_1\boldsymbol{p}_1^\top$ and $\mathbf{Y}_2 = \mathbf{Y}_1 - \boldsymbol{t}_1\boldsymbol{q}_1^\top$, the same procedure is applied iteratively to obtain $\boldsymbol{t}_2, \boldsymbol{p}_2, \boldsymbol{q}_2, \mathbf{X}_3$, and $\mathbf{Y}_3$, and so forth. This iterative process is repeated m times. Consequently, we obtain the decompositions $\mathbf{X} = \boldsymbol{t}_1\boldsymbol{p}_1^\top + \cdots + \boldsymbol{t}_m\boldsymbol{p}_m^\top + \mathbf{X}_{m+1} = \mathbf{T}\mathbf{P}^\top + \mathbf{E}$ and $\mathbf{Y} = \boldsymbol{t}_1\boldsymbol{q}_1^\top + \cdots + \boldsymbol{t}_m\boldsymbol{q}_m^\top + \mathbf{Y}_{m+1} = \mathbf{T}\mathbf{Q}^\top + \mathbf{F}$. This entire process is summarized in Algorithm 1.

2.1.1. Efficient Kernel-Based Methods

Partial Least Squares was initially introduced by Wold (1966) via the NIPALS algorithm, which uses power iteration to solve for $\boldsymbol{v}_k$ and $\boldsymbol{w}_k$ in (2). Höskuldsson (1988) further elucidated the properties of this algorithm. The NIPALS algorithm incurs a substantial computational cost. For simplicity, it is assumed that $l < p$. Assuming an average of a iterations to solve (2), the overall time complexity of the algorithm is $\mathcal{O}(npma)$. Alternatively, Algorithm 1 performs a SVD on $\mathbf{X}^\top\mathbf{Y}$ to solve (2), which also results in a high complexity of $\mathcal{O}(nplm)$. To address this, Lindgren, Geladi, and Wold (1993) proposed a kernel-based approach that computes the covariance matrices $\mathbf{X}^\top\mathbf{X}$ and $\mathbf{X}^\top\mathbf{Y}$ only once, bypassing the high computational burden of the power iteration method and repetitive covariance computation. By substituting the kernel into (2)–(4), the computational cost of these steps is reduced from $\mathcal{O}(npa)$ or $\mathcal{O}(npl)$ to $\mathcal{O}(p^2l)$. For large-scale datasets, the time complexity of the kernel method is dominated by the kernel computation, which is $\mathcal{O}(np^2)$.

Dayal and MacGregor (1997) further enhanced this kernel method by precomputing the covariance matrices and only updating $(\mathbf{X}^\top\mathbf{Y})_k = \mathbf{X}_k^\top\mathbf{Y}_k$ during iterations. The improved kernel method, summarized in Algorithm 2, forms the foundation for the sparsification-based PLS algorithm proposed in this paper.

Besides these kernel approaches, other notable PLS algorithms include SIMPLS (De Jong 1993), OPLS (Trygg and Wold 2002), Envelope methods (Cook, Helland, and Su 2013), 3PRF (Kelly and Pruitt 2015), IDAR (Stocchero 2019), among others. These methods address various challenges in PLS but remain computationally intensive for large-scale data, highlighting the need for more efficient variants.

2.2. Approximating Matrix Multiplication

As shown in Algorithm 2, the computational bottleneck lies in the initial matrix multiplications. Accelerating these operations can significantly reduce the overall time complexity. For general matrices $\mathbf{A}$ and $\mathbf{B}$, a widely used approach is to subsample rows or columns from the matrices to approximate the matrix multiplication (Drineas, Kannan, and Mahoney 2006; Mahoney

Algorithm 1 Partial Least Squares

Input: $\mathbf{X}, \mathbf{Y}$

Initialize: $\mathbf{X}_1 = \mathbf{X}$ and $\mathbf{Y}_1 = \mathbf{Y}$.

for $k \leftarrow 1$ **to** m **do**

  Perform SVD on $\mathbf{X}_k^\top \mathbf{Y}_k$, and extract the left and right singular vectors $\boldsymbol{v}_k$ and $\boldsymbol{w}_k$, respectively, corresponding to the largest singular value. $\mathcal{O}(npl)$

  Compute the latent variable: $\boldsymbol{t}_k = \mathbf{X}_k \boldsymbol{v}_k$. $\mathcal{O}(np)$

  Derive the OLS estimators: $\boldsymbol{p}_k^\top = (\boldsymbol{t}_k^\top \boldsymbol{t}_k)^{-1} \boldsymbol{t}_k^\top \mathbf{X}_k, \quad \boldsymbol{q}_k^\top = (\boldsymbol{t}_k^\top \boldsymbol{t}_k)^{-1} \boldsymbol{t}_k^\top \mathbf{Y}_k$. $\mathcal{O}(np)$

  Update the residual matrices: $\mathbf{X}_{k+1} = \mathbf{X}_k - \boldsymbol{t}_k \boldsymbol{p}_k^\top, \quad \mathbf{Y}_{k+1} = \mathbf{Y}_k - \boldsymbol{t}_k \boldsymbol{q}_k^\top$. $\mathcal{O}(np)$

end

Output: $\mathbf{T}, \mathbf{P}, \mathbf{Q}$

Algorithm 2 Kernel Partial Least Squares

Input: $\mathbf{X}, \mathbf{Y}$

Compute the kernel matrices: $\mathbf{X}^\top \mathbf{X}, \mathbf{X}^\top \mathbf{Y}$ $\mathcal{O}(np^2)$

for $k \leftarrow 1$ **to** m **do**

  Perform an SVD on $(\mathbf{X}^\top \mathbf{Y})_k$ and obtain the left and right singular vectors $\boldsymbol{v}_k$ and $\boldsymbol{w}_k$, respectively, corresponding to the largest singular value. $\mathcal{O}(p^2 l)$

  Compute $\boldsymbol{r}_k$: $\begin{cases} \boldsymbol{r}_1 = \boldsymbol{v}_1, \\ \boldsymbol{r}_k = \boldsymbol{v}_k - \sum_{j=1}^{k-1} \boldsymbol{p}_j^\top \boldsymbol{v}_k \boldsymbol{r}_j, \quad k > 1 \end{cases}$ $\mathcal{O}(p^2)$

  Obtain the OLS estimators: $\boldsymbol{p}_k^\top = \frac{\boldsymbol{r}_k^\top (\mathbf{X}^\top \mathbf{X})_k}{\boldsymbol{r}_k^\top (\mathbf{X}^\top \mathbf{X}) \boldsymbol{r}_k}$, $\boldsymbol{q}_k^\top = \frac{\boldsymbol{r}_k^\top (\mathbf{X}^\top \mathbf{Y})_k}{\boldsymbol{r}_k^\top (\mathbf{X}^\top \mathbf{X}) \boldsymbol{r}_k}$ $\mathcal{O}(p^2)$

  Update the kernel matrix: $(\mathbf{X}^\top \mathbf{Y})_{k+1} = (\mathbf{X}^\top \mathbf{Y})_k - \boldsymbol{p}_k \boldsymbol{q}_k^\top (\boldsymbol{t}_k^\top \boldsymbol{t}_k)$ $\mathcal{O}(pl)$

end

Compute the regression coefficient matrix: $\mathbf{B} = \mathbf{R}\mathbf{Q}^\top$ $\mathcal{O}(mpl)$

Output: $\mathbf{T}, \mathbf{P}, \mathbf{Q}, \mathbf{B}$

2016). A standard algorithm for this approximation is presented in Algorithm 3.

In Algorithm 3, rows (or columns) are sampled based on a predefined probability distribution, and inverse probability weighting is applied to ensure that the resulting matrix product approximates the original product effectively. The quality of this approximation is characterized by the following lemmas.

Lemma 1. *Given matrices* $\mathbf{A}$ *and* $\mathbf{B}$*, the matrices* $\mathbf{C}$ *and* $\mathbf{R}$ *constructed by Algorithm 3 satisfy*

$$\mathbb{E}\left[(\mathbf{CR})_{ij}\right] = (\mathbf{AB})_{ij},$$

$$\mathrm{Var}\left[(\mathbf{CR})_{ij}\right] = \frac{1}{c} \sum_{k=1}^{n} \frac{\mathbf{A}_{ik}^2 \mathbf{B}_{kj}^2}{p_k} - \frac{1}{c} (\mathbf{AB})_{ij}^2.$$

Algorithm 3 Basic Matrix Multiplication Approximation Algorithm

Input: Matrices $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{B} \in \mathbb{R}^{n\times l}$; sampling probabilities $\{p_i\}_{i=1}^n$.

1 **for** $t \leftarrow 1$ **to** c **do**

2 Sample an index $i_t \in \{1, \ldots, n\}$ with replacement, according to the probability distribution $\Pr[i_t = k] = p_k$.

3 Set $\boldsymbol{c}_t = \boldsymbol{a}_{i_t}/\sqrt{cp_{i_t}}, \boldsymbol{r}_{(t)} = \boldsymbol{b}_{(i_t)}/\sqrt{cp_{i_t}}$

4 **end** $\mathcal{O}(n + cm + cl)$

Output: Matrix multiplication $\mathbf{CR}$ such that $\mathbf{CR} \approx \mathbf{AB}$, where $\mathbf{C} \in \mathbb{R}^{m\times c}$ and $\mathbf{R} \in \mathbb{R}^{c\times l}$ $\mathcal{O}(cml)$

Lemma 1 indicates that $\mathbf{CR}$ is an unbiased estimator of $\mathbf{AB}$, with its variance affected by the sampling probabilities in Algorithm 3. Consequently, adjusting these sampling probabilities directly influences the accuracy of the approximation. To further analyze this, Drineas, Kannan, and Mahoney (2006) consider the sum of the element-wise variances, which is equivalent to the expected squared Frobenius norm of the approximation error, denoted by $\mathbb{E}[\|\mathbf{AB} - \mathbf{CR}\|_F^2]$. The following lemma quantifies this relationship.

Lemma 2. Given matrices $\mathbf{A}$ *and* $\mathbf{B}$*, the matrices* $\mathbf{C}$ *and* $\mathbf{R}$ *constructed by Algorithm 3 satisfy*

$$\mathbb{E}\left[\|\mathbf{AB} - \mathbf{CR}\|_F^2\right] = \sum_{k=1}^{n} \frac{\|\boldsymbol{a}_k\|_2^2 \left\|\boldsymbol{b}_{(k)}\right\|_2^2}{cp_k} - \frac{1}{c}\|\mathbf{AB}\|_F^2.$$

When the sampling probability is set to $p_k = \frac{\|\boldsymbol{a}_k\|_2\|\boldsymbol{b}_{(k)}\|_2}{\sum_{k'=1}^{n}\|\boldsymbol{a}_{k'}\|_2\|\boldsymbol{b}_{(k')}\|_2}$, $\mathbb{E}[\|\mathbf{AB} - \mathbf{CR}\|_F^2]$ *attains its minimum value*

$$\mathbb{E}\left[\|\mathbf{AB} - \mathbf{CR}\|_F^2\right] = \frac{1}{c}\left(\sum_{k=1}^{n}\|\boldsymbol{a}_k\|_2\left\|\boldsymbol{b}_{(k)}\right\|_2\right)^2 - \frac{1}{c}\|\mathbf{AB}\|_F^2. \quad (5)$$

Proofs of Lemma 1 and Lemma 2 can be found in Drineas, Kannan, and Mahoney (2006) and Mahoney (2016). In contrast to row (or column) sampling, element-wise sampling provides an alternative approach that focuses on directly approximating the original matrix itself, rather than the resulting product.

The operator nnz denotes the number of non-zero elements in a matrix. By carefully selecting the probabilities p_{ij}, Algorithm 4 can ensure that $\|\mathbf{A}^* - \mathbf{A}\|_2$ is effectively controlled (Braverman et al. 2021). Applying Algorithm 4 yields sketches $\mathbf{A}^*$ and $\mathbf{B}^*$ of $\mathbf{A}$ and $\mathbf{B}$, respectively, such that their product $\mathbf{A}^*\mathbf{B}^*$ is an unbiased estimator of $\mathbf{AB}$. As shown in Drineas, Kannan, and Mahoney (2006), specific choices of probability values can further provide an upper bound on the variance of $\mathbf{A}^*\mathbf{B}^*$. However, identifying the optimal probabilities that minimize the variance of this estimator remains an open question.

In Section 3, we extend the fundamental idea of Algorithm 4 to approximate matrix multiplications, aiming to inherit some of the desirable properties exhibited by Algorithm 3 while addressing its limitations.

2.3. Core-Elements Estimator for Linear Models

Following the notation in Model (1), consider a general linear regression model:

$$\mathbf{Y} = \mathbf{XB} + \mathbf{E}, \quad (6)$$

the ordinary least squares estimator for $\mathbf{B}$ is given by:

$$\widehat{\mathbf{B}} = \left(\mathbf{X}^\top\mathbf{X}\right)^{-1}\mathbf{X}^\top\mathbf{Y}, \quad (7)$$

which is analogous to the estimator used in (4). For large-scale datasets, the computational cost of the OLS estimator is dominated by the computation of $\mathbf{X}^\top\mathbf{X}$ and $\mathbf{X}^\top\mathbf{Y}$, with a time complexity of $\mathcal{O}(np^2 + npl)$. To mitigate this, classical row subsampling techniques reduce the time complexity to $\mathcal{O}(rp^2 + rpl)$ by selecting only r rows of data, as outlined in Algorithm 3.

Beyond row subsampling, element-wise subsampling offers an alternative for reducing computational complexity. Given the predictor matrix $\mathbf{X}$, we can obtain a sparse sketch $\mathbf{X}^*$ using Algorithm 4. If $\mathbf{X}^*$ contains rp nonzero elements (analogous to row subsampling), the time complexity for computing $\mathbf{X}^{*\top}\mathbf{X}$ and $\mathbf{X}^{*\top}\mathbf{Y}$ can also be reduced to $\mathcal{O}(rp^2 + rpl)$.

With this sparse sketch $\mathbf{X}^*$, Li et al. (2024) proposed the Core-Elements estimator, which takes the form:

$$\tilde{\mathbf{B}} = \left(\mathbf{X}^{*\top}\mathbf{X}\right)^{-1}\mathbf{X}^{*\top}\mathbf{Y}. \quad (8)$$

A key advantage of this estimator is its unbiasedness. Under the assumptions in (6), and assuming $\mathbf{X}^{*\top}\mathbf{X}$ is invertible, the expectation of $\tilde{\mathbf{B}}$ is:

$$\mathbb{E}(\tilde{\mathbf{B}}) = \mathbb{E}\left(\left(\mathbf{X}^{*\top}\mathbf{X}\right)^{-1}\mathbf{X}^{*\top}\mathbf{Y}\right) = \left(\mathbf{X}^{*\top}\mathbf{X}\right)^{-1}\mathbf{X}^{*\top}$$

$$\mathbb{E}(\mathbf{Y}) = \left(\mathbf{X}^{*\top}\mathbf{X}\right)^{-1}\mathbf{X}^{*\top}\mathbf{XB} = \mathbf{B}.$$

Therefore, $\tilde{\mathbf{B}}$ is an unbiased estimator of $\mathbf{B}$. Notably, this unbiasedness does not impose any restrictions on the choice of

Algorithm 4 Element-wise Matrix Approximation Algorithm

Input: Matrix $\mathbf{A}_{m\times n}$, sampling probabilities $\{p_{ij}\}_{i,j=1}^{m,n}$.

```
for i ← 1 to m do
    for j ← 1 to n do
        Independently set A*_ij to A_ij / p_ij with probability p_ij, and to 0 otherwise.
    end
end                                                          O(nnz(A))
```

Output: Matrix $\mathbf{A}^*_{m\times n}$

$\mathbf{X}^*$. Any matrix $\mathbf{X}^*$, not just those derived from sparsification schemes, will lead to an unbiased estimator $\tilde{\mathbf{B}}$. This flexibility significantly broadens the applicability and versatility of the Core-Elements estimator.

3. Methods

This section introduces the core algorithm of this paper. First, we present a matrix multiplication approximation method based on element-wise subsampling, along with optimal sampling probabilities to minimize estimation variance. Subsequently, we integrate this sparse sketch into the Partial Least Squares regression algorithm, reducing its computational complexity. Finally, we derive the regression coefficients, and propose the improved sparsification algorithm.

3.1. Element-Wise Approximation for Matrix Multiplication

To approximate the matrix product $\mathbf{AB}$, where $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times l}$, unlike prior work (Drineas, Kannan, and Mahoney 2006), which sparsifies both matrices, we only sparsify $\mathbf{A}$ while retaining the full matrix $\mathbf{B}$ inspired by the Core-Elements estimator in (8). This approach effectively reduces time complexity and has favorable approximation properties.

Specifically, we use Algorithm 4 to construct a sparse matrix $\mathbf{A}^*$ and approximate $\mathbf{AB}$ with $\mathbf{A}^*\mathbf{B}$. Given that $\sum_{i,j} p_{ij} = mc$, where mc denotes the expected number of nonzero elements in $\mathbf{A}^*$, the time complexity of computing $\mathbf{A}^*\mathbf{B}$ is $\mathcal{O}(cml)$, consistent with that of row-wise subsampling in Algorithm 3. Similar to Lemma 1 and Lemma 2, the following theorems detail the approximation properties of $\mathbf{A}^*\mathbf{B}$.

Theorem 1 (Unbiasedness and Variance). *Given matrices* $\mathbf{A}$ *and* $\mathbf{B}$*, the matrix* $\mathbf{A}^*$ *obtained from Algorithm 4 satisfies*

$$\mathbb{E}\left[\left(\mathbf{A}^*\mathbf{B}\right)_{ij}\right] = (\mathbf{AB})_{ij},$$

$$\mathrm{Var}\left[\left(\mathbf{A}^*\mathbf{B}\right)_{ij}\right] = \sum_{k=1}^{n}\left(\left(\frac{1}{p_{ik}} - 1\right)\mathbf{A}_{ik}^2\mathbf{B}_{kj}^2\right).$$

Theorem 2 (Expected Approximation Error). *Given matrices* $\mathbf{A}$ *and* $\mathbf{B}$*, the expected Frobenius norm of the error in approximating* $\mathbf{AB}$ *using* $\mathbf{A}^*\mathbf{B}$ *is*

$$\mathbb{E}\left[\left\|\mathbf{A}^*\mathbf{B} - \mathbf{AB}\right\|_F^2\right] = \sum_{i=1}^{m}\sum_{k=1}^{n}\left(\left(\frac{1}{p_{ik}} - 1\right)\mathbf{A}_{ik}^2\left\|\boldsymbol{b}_{(k)}\right\|_2^2\right).$$

When the sampling probability is set to $p_{ik} \propto |\mathbf{A}_{ik}|\|\boldsymbol{b}_{(k)}\|_2$ *attains its minimum value*

$$\mathbb{E}\left[\left\|\mathbf{A}^*\mathbf{B} - \mathbf{AB}\right\|_F^2\right] = \frac{1}{mc}\left(\sum_{i=1}^{m}\sum_{k=1}^{n}|\mathbf{A}_{ik}|\left\|\boldsymbol{b}_{(k)}\right\|_2\right)^2 - \sum_{i=1}^{m}\sum_{k=1}^{n}\left(\mathbf{A}_{ik}^2\left\|\boldsymbol{b}_{(k)}\right\|_2^2\right),$$

where $\sum_{i=1}^{m}\sum_{k=1}^{n} p_{ik} = mc$.

Remark 1. Algorithm 4 can achieve variance reduction comparable to that of Algorithm 3. Specifically, when the element-wise sampling probability p_{ij} for each row in Algorithm 4 is set to the row sampling probability p_i in Algorithm 3, it can be shown that

$$\mathbb{E}[\|\mathbf{A}^*\mathbf{B} - \mathbf{AB}\|_F^2] = \sum_{k=1}^{n}\frac{\|\boldsymbol{a}_k\|_2^2\left\|\boldsymbol{b}_{(k)}\right\|_2^2}{p_k} - \sum_{i=1}^{m}\sum_{k=1}^{n}\left(\mathbf{A}_{ik}^2\left\|\boldsymbol{b}_{(k)}\right\|_2^2\right),$$

and it attains its minimum value

$$\mathbb{E}[\|\mathbf{A}^*\mathbf{B} - \mathbf{AB}\|_F^2] = \frac{1}{c}\left(\sum_{k=1}^{n}\|\boldsymbol{a}_k\|_2\left\|\boldsymbol{b}_{(k)}\right\|_2\right)^2 - \|\mathbf{AB}\|_F^2, \quad (9)$$

where $p_k \propto \|\boldsymbol{a}_k\|_2\|\boldsymbol{b}_{(k)}\|_2$ and $\sum_{k=1}^{n} p_k = c$. Notably, when $c = 1$, (9) is equivalent to (5). As c increases, (9) decreases faster than (5) because only the first term in (9) is inversely proportional to c. Furthermore, (9) can be viewed as a special case of Theorem 2 when the constraint $p_{1k} = p_{2k} = \cdots = p_{mk}, \forall k$ is imposed on the sampling probabilities in Theorem 2. By the principles of constrained optimization, we have

$$\mathbb{E}\left[\left\|\mathbf{A}^*\mathbf{B} - \mathbf{AB}\right\|_F^2\right] \le \mathbb{E}\left[\|\mathbf{AB} - \mathbf{CR}\|_F^2\right],$$

where $\mathbf{A}^*$ is the matrix obtained from Algorithm 4 with the optimal probability in Theorem 2, and $\mathbf{C}$ and $\mathbf{R}$ are the matrices obtained from Algorithm 3 with the optimal probability in Lemma 2. This implies that the approximation $\mathbf{A}^*\mathbf{B}$ obtained

via element-wise sampling in Algorithm 4 and Theorem 2 will generally exhibit a lower variance compared to **CR** obtained from row sampling in Algorithm 3 and Lemma 2.

Theorems 1 and 2 demonstrate that element-wise sampling approximation from Algorithm 4 preserves the unbiasedness and generally has lower variance than the row-sampling-based approximation. Furthermore, by introducing projection transformations to the matrices involved in the multiplication, we can derive an even stronger result.

Theorem 3. Given matrices $\mathbf{A}$ *and* $\mathbf{B}$*, and any projection matrices* $\mathbf{S}_1$ *and* $\mathbf{S}_2$ *of appropriate dimensions, consider the approximation of the matrix* $\mathbf{S}_1^\top \mathbf{ABS}_2$*. The* $\mathbf{A}^*$ *obtained by Algorithm 4 satisfies:*

$$\mathbb{E}\left[\left\|\mathbf{S}_1^\top \mathbf{A}^*\mathbf{BS}_2 - \mathbf{S}_1^\top \mathbf{ABS}_2\right\|_F^2\right] = \sum_{i=1}^{n}\sum_{k=1}^{m}\left(\boldsymbol{b}_{(i)}^\top \mathbf{D}\boldsymbol{b}_{(i)}\mathbf{C}_{kk}\mathbf{A}_{ki}^2\left(\frac{1}{p_{ki}}-1\right)\right).$$

Furthermore, when the sampling probability is set to $p_{ki} \propto |\mathbf{A}_{ki}|\sqrt{\mathbf{C}_{kk}\boldsymbol{b}_{(i)}^\top \mathbf{D}\boldsymbol{b}_{(i)}}$*, the above expression attains its minimum value*

$$\mathbb{E}\left[\left\|\mathbf{S}_1^\top \mathbf{A}^*\mathbf{BS}_2 - \mathbf{S}_1^\top \mathbf{ABS}_2\right\|_F^2\right] = \frac{1}{mc}\left(\sum_{i=1}^{m}\sum_{k=1}^{n}|\mathbf{A}_{ki}|\sqrt{\mathbf{C}_{kk}\boldsymbol{b}_{(i)}^\top \mathbf{D}\boldsymbol{b}_{(i)}}\right)^2 - \sum_{i=1}^{n}\sum_{k=1}^{m}\left(\boldsymbol{b}_{(i)}^\top \mathbf{D}\boldsymbol{b}_{(i)}\mathbf{C}_{kk}\mathbf{A}_{ki}^2\right),$$

where $\mathbf{C} = \mathbf{S}_1\mathbf{S}_1^\top, \mathbf{D} = \mathbf{S}_2\mathbf{S}_2^\top$ *and* $\sum_{i=1}^{m}\sum_{k=1}^{n} p_{ik} = mc$*.*

Remark 2. The optimal sampling probabilities derived in Theorem 2 and Theorem 3 have the form $p_{ij} = \frac{|\mathbf{A}_{ij}|}{f_j g_i}$, where $f_j \propto \mathbf{C}_{jj}^{-\frac{1}{2}}$ depends only on the index j and $g_i \propto (\boldsymbol{b}_{(i)}^\top \mathbf{D}\boldsymbol{b}_{(i)})^{-\frac{1}{2}}$ depends only on the index i. Consequently, the elements of $\mathbf{A}^*$ obtained via Algorithm 4 can be expressed as:

$$\mathbf{A}_{ij}^* = \begin{cases} \frac{\mathbf{A}_{ij}}{|\mathbf{A}_{ij}|} f_j g_i, & \text{with probability } p_{ij}, \\ 0, & \text{otherwise.} \end{cases}$$

Defining $\boldsymbol{f} = (f_1, \ldots, f_p), \boldsymbol{g} = (g_1, \ldots, g_n), \mathbf{A}^{*\prime} = (\text{sign}(\mathbf{A}_{ij}))_{n\times p}$, we can decompose $\mathbf{A}^*$ as:

$$\mathbf{A}^* = (\boldsymbol{f}\boldsymbol{g}^\top) \odot \mathbf{A}^{*\prime}.$$

As shown in Figure 2, assuming the elements of $\mathbf{A}$ are positive for simplicity, the resulting $\mathbf{A}^{*\prime}$ is a binary matrix with elements in $\{0, 1\}$. This decomposition reduces storage requirements and facilitates faster matrix multiplication operations.

Building upon Theorem 2 and Theorem 3, several specific sampling probability schemes are now introduced.

Corollary 1 (Efficient Sampling Schemes). *For a matrix* $\mathbf{X}$*, the matrix* $\mathbf{X}^*$ *obtained via Algorithm 4 satisfies:*

(i) *(Covariance Approximation) When the sampling probability is set to* $p_{ij} = \frac{\beta|x_{ij}|\|x_i\|_2}{\sum_{i=1}^{n}\sum_{j=1}^{j}|x_{ij}|\|x_i\|_2}$*, the expression* $\mathbb{E}[\|\mathbf{X}^{*\top}\mathbf{X} - \mathbf{X}^\top\mathbf{X}\|_F^2]$ *achieves its minimum value.*

(ii) *(Cross-Covariance Approximation) When the sampling probability is set to* $p_{ij} = \frac{\beta|x_{ij}|\|y_i\|_2}{\sum_{i=1}^{n}\sum_{j=1}^{j}|x_{ij}|\|y_i\|_2}$*, the expression* $\mathbb{E}[\|\mathbf{X}^{\star\top}\mathbf{Y} - \mathbf{X}^\top\mathbf{Y}\|_F^2]$ *achieves its minimum value.*

(iii) *(Projection Approximation) Given the SVD decomposition* $\mathbf{X} = \mathbf{U}\Lambda\mathbf{V}^\top$*, and defining the projection matrix* $\mathbf{S} = \Lambda^{-\frac{1}{2}}\mathbf{V}$*, when the sampling probability is set to* $p_{ij} = \frac{\beta|x_{ij}|\sqrt{h_{ii}}}{\sum_{i=1}^{n}\sum_{j=1}^{j}|x_{ij}|\sqrt{h_{ii}}}$*, the expression* $\mathbb{E}[\|\mathbf{X}^{*\top}\mathbf{XS} - \mathbf{X}^\top\mathbf{XS}\|_F^2]$ *achieves its minimum value, where* $h_{ii} = x_i^\top(\mathbf{X}^\top\mathbf{X})^{-1}x_i$ *is known as the leverage score.*

3.2. Sparsification Subsampling for PLS

3.2.1. Integration of the Sparse Sketch

The sparse sketch $\mathbf{X}^*$ of the observation matrix $\mathbf{X}$ is constructed using Algorithm 4 and Corollary 1. In the context of PLS, we primarily employ the covariance approximation detailed in Corollary 1 to construct the sparse sketch, as $\mathbf{X}^\top\mathbf{X}$ plays a central role in the algorithm. To accelerate the PLS regression, the primary task involves applying this sparse matrix to the most computationally intensive operations, namely $\mathbf{X}^{*\top}\mathbf{X}$ and $\mathbf{X}^{*\top}\mathbf{Y}$. However, given the direct involvement of $\mathbf{X}$ and $\mathbf{Y}$ in Algorithm 1, a coherent rationale is required to justify the integration of $\mathbf{X}^*$ within the PLS algorithm.

A regression assumption analogous to (1) is considered:

$$\begin{cases} \mathbf{X} = \mathbf{TP}^\top + \mathbf{E}, \\ \mathbf{X}^* = \mathbf{T}^*\mathbf{P}^\top + \mathbf{E}^*, \\ \mathbf{Y} = \mathbf{TQ}^\top + \mathbf{F}. \end{cases}$$

Here, it is postulated that $\mathbf{X}$ and $\mathbf{X}^*$ share the same parameter matrix $\mathbf{P}$ (this assumption is validated in the proof of Theorem 4). To reduce computational complexity, we replace the computation of $\mathbf{X}^\top\mathbf{Y}$ in (2) with $\mathbf{X}^{*\top}\mathbf{Y}$ for the derivation of parameters $\boldsymbol{u}_1$ and $\boldsymbol{v}_1$. Since $\mathbf{X}^*$ and $\mathbf{X}$ share parameters, it follows that $\boldsymbol{t}_1 = \mathbf{X}\boldsymbol{v}_1$ and $\boldsymbol{t}_1^* = \mathbf{X}^*\boldsymbol{v}_1$. While only $\boldsymbol{t}_1$ is employed in (3) for estimating $\mathbf{X}$ and $\mathbf{Y}$, the introduction of $\boldsymbol{t}_1^*$ motivates the use of the Core-Elements estimation (8) for parameters $\boldsymbol{p}_1$ and $\boldsymbol{q}_1$:

$$\begin{cases} \boldsymbol{p}_1^\top = \left(\boldsymbol{t}_1^{*\top}\boldsymbol{t}_1\right)^{-1}\boldsymbol{t}_1^{*\top}\mathbf{X}, \\ \boldsymbol{q}_1^\top = \left(\boldsymbol{t}_1^{*\top}\boldsymbol{t}_1\right)^{-1}\boldsymbol{t}_1^{*\top}\mathbf{Y}. \end{cases} \tag{10}$$

The residual terms are subsequently derived as follows:

$$\begin{cases} \mathbf{X}_2 = \mathbf{X}_1 - \boldsymbol{t}_1\boldsymbol{p}_1^\top, \\ \mathbf{X}_2^* = \mathbf{X}_1^* - \boldsymbol{t}_1^*\boldsymbol{p}_1^\top, \\ \mathbf{Y}_2 = \mathbf{Y}_1 - \boldsymbol{t}_1\boldsymbol{q}_1^\top. \end{cases} \tag{11}$$

This iterative process computes subsequent terms $\boldsymbol{t}_2, \boldsymbol{t}_2^*, \boldsymbol{p}_2, \boldsymbol{q}_2, \mathbf{X}_3, \mathbf{X}_3^*, \mathbf{Y}_3$ and so on, up to m iterations, resulting in a sparsified PLS algorithm based on $\mathbf{X}^*$.

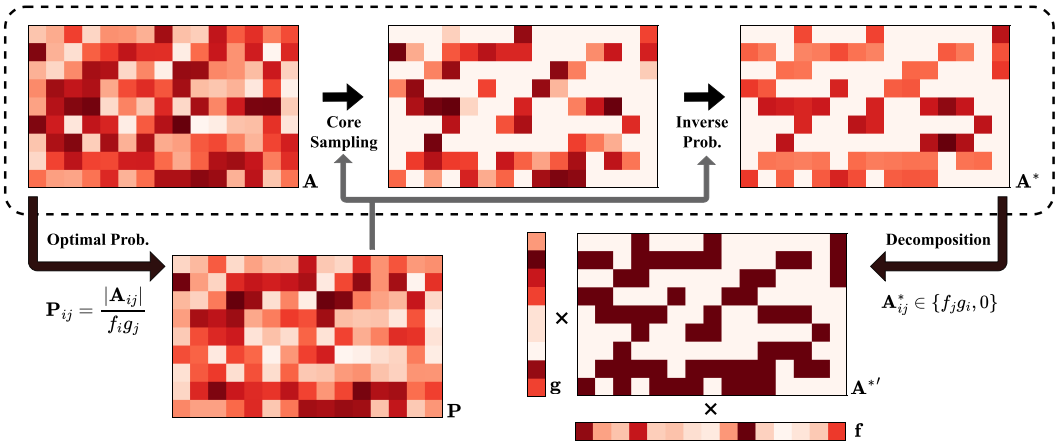


Figure 2. Illustration of algorithm 4 and remark 2, demonstrating how the decomposition $\mathbf{A}^* = (\boldsymbol{f}\boldsymbol{g}^\top) \odot \mathbf{A}^{*\prime}$ leads to reduced storage requirements. Consequently, the resulting $\mathbf{A}^{*\prime}$ and $\boldsymbol{f}, \boldsymbol{g}$ have more compact representations compared to a general $\mathbf{A}^*$.

3.2.2. Reducing Time Complexity with Kernel Updates

However, the matrix update operations in (11) still incur a complexity of $\mathcal{O}(np)$ per iteration, which results in a total complexity of $\mathcal{O}(npm)$. To address this, we employ the kernel-based PLS algorithm (Algorithm 2), which avoids direct updates $\mathbf{X}_k$, $\mathbf{X}_k^*$, and $\mathbf{Y}_k$. Instead, we update $\mathbf{X}_k^{*\top}\mathbf{X}_k$ and $\mathbf{X}_k^{*\top}\mathbf{Y}_k$ directly.

During the k-th iteration of the algorithm, the computation of $\boldsymbol{u}_k$ and $\boldsymbol{v}_k$ requires an SVD of $\mathbf{X}_{k-1}^{*\top}\mathbf{Y}_{k-1}$, followed by the derivation of $\boldsymbol{p}_k$ and $\boldsymbol{q}_k$ using $\boldsymbol{t}_k$ and $\boldsymbol{t}_k^*$. Parameters $\boldsymbol{p}_k$ and $\boldsymbol{q}_k$ can be reformulated as

$$\begin{cases} \boldsymbol{p}_k^\top = (\boldsymbol{t}_k^{*\top}\boldsymbol{t}_k)^{-1}\boldsymbol{t}_k^{*\top}\mathbf{X}_k = \dfrac{\boldsymbol{v}_k^\top \mathbf{X}_k^{*\top}\mathbf{X}_k}{\boldsymbol{v}_k^\top \mathbf{X}_k^{*\top}\mathbf{X}_k\boldsymbol{v}_k}, \\ \boldsymbol{q}_k^\top = (\boldsymbol{t}_k^{*\top}\boldsymbol{t}_k)^{-1}\boldsymbol{t}_k^{*\top}\mathbf{Y}_k = \dfrac{\boldsymbol{v}_k^\top \mathbf{X}_k^{*\top}\mathbf{Y}_k}{\boldsymbol{v}_k^\top \mathbf{X}_k^{*\top}\mathbf{X}_k\boldsymbol{v}_k}. \end{cases} \tag{12}$$

We can replace explicit matrix updates in (11) with updates to $\mathbf{X}_k^{*\top}\mathbf{X}_k$ and $\mathbf{X}_k^{*\top}\mathbf{Y}_k$. This is formalized in Theorem 4. Notably, $\mathbf{X}^*$ and $\mathbf{X}$ continue to share the same parameter $\boldsymbol{p}_1$. Indeed, as demonstrated in the proof of Theorem 4, even if the corresponding parameter $\boldsymbol{p}^*$ for $\mathbf{X}^*$ is estimated using alternative formulations, the algorithmic outcomes remain invariant.

Theorem 4. Assuming $\mathbf{X}^* = \mathbf{T}^*\mathbf{P}^{*\top} + \mathbf{E}^*$, the iterative update formulas for the parameters within the kernel algorithm framework, independent of the specific value assigned to $\mathbf{P}^*$, are expressed as follows:

$$\begin{cases} \mathbf{X}_{k+1}^{*\top}\mathbf{X}_{k+1} = \mathbf{X}_k^{*\top}\mathbf{X}_k\left(\mathbf{I} - \boldsymbol{v}_k\boldsymbol{p}_k^\top\right), \\ \mathbf{X}_{k+1}^{*\top}\mathbf{Y}_{k+1} = \mathbf{X}_k^{*\top}\mathbf{Y}_k - \boldsymbol{p}_k'\boldsymbol{q}_k^\top\left(\boldsymbol{t}_k^{*\top}\boldsymbol{t}_k\right), \end{cases}$$

where $\boldsymbol{p}_k' = \dfrac{\mathbf{X}_k^{*\top}\mathbf{X}_k\boldsymbol{v}_k}{\boldsymbol{v}_k^\top \mathbf{X}_k^{*\top}\mathbf{X}_k\boldsymbol{v}_k}$.

Theorem 4 establishes that the introduction of the kernel algorithm renders the parameter iteration dependent solely on the data dimensions p and l, independent of the sample size n. This underscores the potential of kernel algorithms to significantly reduce time complexity in large-scale data scenarios. Moreover, the parameter iteration formula is shown to be independent of $\mathbf{P}^*$, thereby justifying the assumption of a shared parameter $\mathbf{P}$ for both $\mathbf{X}^*$ and $\mathbf{X}$ in (10).

In summary, we propose a kernel-based PLS algorithm incorporating sparsification subsampling, ensuring that the primary time complexity is dominated by the initial kernel computation step.

3.2.3. Improved Algorithm for Regression Coefficients

While the sparsification subsampling PLS algorithm based on kernel updating yields the PLS loading parameters $\mathbf{P}$ and $\mathbf{Q}$, it does not directly provide the PLS regression coefficients $\mathbf{B}$ in (6). To circumvent the need for additional computation to obtain $\mathbf{B}$, we have developed an improved kernel method inspired by Algorithm 2 (Dayal and MacGregor 1997). This method also contributes to further acceleration of the algorithm. For clarity in the subsequent discussion, let $\mathbf{T} = \mathbf{XR}$, which implies $\mathbf{B} = \mathbf{RQ}^\top$. Consequently, computing $\mathbf{R}$ during the algorithm suffices to readily obtain $\mathbf{B}$.

As established in Section 2, the k-th iteration involves $\boldsymbol{t}_k = \mathbf{X}_k\boldsymbol{v}_k$. Combining this with (11) yields:

$$\boldsymbol{t}_k = \mathbf{X}_k\boldsymbol{v}_k = \left(\mathbf{X}_{k-1} - \boldsymbol{t}_{k-1}\boldsymbol{p}_{k-1}^\top\right)\boldsymbol{v}_k = \mathbf{X}_{k-1}\left(\mathbf{I} - \boldsymbol{v}_{k-1}\boldsymbol{p}_{k-1}^\top\right)$$

$$\boldsymbol{v}_k = \mathbf{X}\prod_{i=1}^{k-1}\left(\mathbf{I} - \boldsymbol{v}_i\boldsymbol{p}_i^\top\right)\boldsymbol{v}_k.$$

Introducing $\mathbf{H}_{k-1} = \prod_{i=1}^{k-1}(\mathbf{I} - \boldsymbol{v}_i\boldsymbol{p}_i^\top)$, the following is obtained based on the preceding derivation:

$$\boldsymbol{r}_k = \prod_{i=1}^{k-1}\left(\mathbf{I} - \boldsymbol{v}_i\boldsymbol{p}_i^\top\right)\boldsymbol{v}_k = \mathbf{H}_{k-1}\boldsymbol{v}_k.$$

Importantly, the introduction of $\mathbf{H}_k$ enables the derivation of the update formulas for $\mathbf{X}_k^{*\top}\mathbf{X}_k$ and $\mathbf{X}_k^{*\top}\mathbf{Y}_k$.

Theorem 5. Given the computed variable $\mathbf{H}_k$, the updates for $\mathbf{X}_k^{*\top}\mathbf{X}_k$ and $\mathbf{X}_k^{*\top}\mathbf{Y}_k$ can be expressed as follows:

$$\begin{cases} \mathbf{X}_{k+1}^{*\top}\mathbf{X}_{k+1} = \mathbf{X}^{*\top}\mathbf{X}\mathbf{H}_k, \\ \mathbf{X}_{k+1}^{*\top}\mathbf{Y}_{k+1} = \mathbf{X}_k^{*\top}\mathbf{Y}_k - \boldsymbol{p}_k'\boldsymbol{q}_k^\top\left(\boldsymbol{t}_k^{*\top}\boldsymbol{t}_k\right). \end{cases}$$

Notably, the introduction of variables $\boldsymbol{r}_k$ and $\mathbf{H}_k$ optimizes the update process for $\mathbf{X}_k^{*\top}\mathbf{X}_k$. The improved algorithm is summarized as follows.

The time complexity of Algorithm 5 is analyzed as follows. For simplicity, it is assumed that $l < p$. The first step, involving sparse subsampling of $\mathbf{X}$, has a time complexity of $\mathcal{O}(\text{nnz}(\mathbf{X}))$. The second step, which performs matrix multiplication, has a

Algorithm 5 Sparsification subsampling for improved kernel algorithm

Input: $\mathbf{X}, \mathbf{Y}$

Compute the sparse projection $\mathbf{X}^*$ of $\mathbf{X}$ using Algorithm 4, the sampling probability is given by Corollary 1 $\mathcal{O}(\text{nnz}(\mathbf{X}))$

Compute the approximate covariance matrices $\mathbf{X}^{*\top}\mathbf{X}$ and $\mathbf{X}^{*\top}\mathbf{Y}$ $\mathcal{O}(rp^2)$

for $k \leftarrow 1$ **to** m **do**

Perform an SVD decomposition on $\mathbf{X}_k^{*\top}\mathbf{Y}_k$ and find the vectors $\boldsymbol{v}_k$ and $\boldsymbol{w}_k$ corresponding to the largest singular value. $\mathcal{O}(p^2l)$

$\boldsymbol{r}_k = \mathbf{H}_{k-1}\boldsymbol{v}_k, \mathbf{H}_k = \prod_{i=1}^{k}\left(\mathbf{I} - \boldsymbol{v}_i\boldsymbol{p}_i^\top\right)$ $\mathcal{O}(p^2)$

$\boldsymbol{p}_k^\top = \frac{\boldsymbol{v}_k^\top \mathbf{X}^{*\top}\mathbf{X}\mathbf{H}_k}{\boldsymbol{v}_k^\top \mathbf{X}^{*\top}\mathbf{X}\boldsymbol{r}_k}, \quad \boldsymbol{p}_k'^\top = \frac{\boldsymbol{r}_k^\top \mathbf{X}^\top \mathbf{X}^*}{\boldsymbol{v}_k^\top \mathbf{X}^{*\top}\mathbf{X}\boldsymbol{r}_k}, \quad \boldsymbol{q}_k^\top = \frac{\boldsymbol{v}_k^\top \mathbf{X}_k^{*\top}\mathbf{Y}_k}{\boldsymbol{v}_k^\top \mathbf{X}^{*\top}\mathbf{X}\boldsymbol{r}_k}.$ $\mathcal{O}(p^2)$

$\mathbf{X}_{k+1}^{*\,\top}\mathbf{Y}_{k+1} = \mathbf{X}_k^{*\,\top}\mathbf{Y}_k - \boldsymbol{p}_k'\boldsymbol{q}_k^\top\left(\boldsymbol{t}_k^{*\,\top}\boldsymbol{t}_k\right)$ $\mathcal{O}(pl)$

end

$\mathbf{B} = \mathbf{R}\mathbf{Q}^\top$ $\mathcal{O}(mpl)$

Output: $\mathbf{T}, \mathbf{P}, \mathbf{Q}, \mathbf{B}$

time complexity of $\mathcal{O}(rp^2)$. The iterative process of solving the parameters is primarily governed by the SVD computation in each iteration, resulting in a complexity of $\mathcal{O}(mp^2l)$. Consequently, the overall time complexity is $\mathcal{O}(\text{nnz}(\mathbf{X}) + rp^2 + mp^2l)$. In the context of large-scale data, where $n \gg r \gg p$ to mitigate significant performance degradation, the time complexity can be reduced to $\mathcal{O}(rp^2)$. The number of iterations, m, is typically determined through cross-validation in preliminary experiments. In most scenarios, only a low-rank latent variable is considered, that is, $m \ll p$.

4. Simulation Studies

In this section, we evaluate the performance of our proposed algorithm through a Monte Carlo simulation study. We compare Algorithm 5 (SPAR, with the sampling probability defined as in Corollary 1 (i)) with five representative sampling-based competitors: uniform row sampling (UNIF), leverage score-based sampling (LEV), influence function-based sampling (IF) (Xie, Feng, and Chen 2022), minimum covariance determinant-based subset selection (MCD) (Huang et al. 2024), and the full-sample approach (FULL). The main text primarily focuses on sampling-based methods, since they are the most directly comparable to SPAR in terms of reducing the computational burden by using only a subset of the information from the predictor matrix. For completeness, Table 1 summarizes the computational costs, method types, and acceleration strategies of several classical and contemporary PLS algorithms, including online or stochastic PLS variants. Additional experiments providing broader comparisons with these methods are reported in Section C of the Supplementary Material.

To ensure a fair comparison among sampling-based methods, we sample r rows for row-sampling methods and select rp elements for SPAR, so that the number of nonzero elements used from the predictor matrix is comparable across different sampling strategies. All experiments were conducted using R on a machine with a processor clock speed of 2.20 GHz and 32 GB of memory.

We generate $\mathbf{X}$ and $\mathbf{Y}$ by randomly generating the latent variable $\mathbf{T}$, under the experimental assumptions specified in (1):

$$\begin{cases} \mathbf{X} = \mathbf{T}\mathbf{P}^\top + \mathbf{E}, \\ \mathbf{Y} = \mathbf{T}\mathbf{Q}^\top + \mathbf{F}, \end{cases} \tag{13}$$

where $\mathbf{P}$ and $\mathbf{Q}$ are randomly generated orthogonal matrices, and $\mathbf{T}$ follows the distribution:

$$\boldsymbol{t}_{(i)} \sim N_m\left(0, \Sigma_1\Sigma_2\right), \quad \Sigma_1 = \left(0.5^{|i-j|}\right)_{ij}$$
$$\Sigma_2 = \text{diag}\left(1, 20^{\frac{1}{m-1}}, 20^{\frac{2}{m-1}}, \ldots, 20\right). \tag{14}$$

Concurrently, $\mathbf{E}$ and $\mathbf{F}$ are residual matrices satisfying:

$$\mathbf{E}_{ij} \sim N\left(0, \sigma_1^2\right), \quad \mathbf{F}_{\text{ij}} \sim N\left(0, \sigma_2^2\right).$$

The variances σ_1^2 and σ_2^2 are determined by the signal-to-noise ratios, defined by $\gamma_1 = \frac{\text{Var}(\mathbf{T}\mathbf{P}^\top)}{\sigma_1^2}, \gamma_2 = \frac{\text{Var}(\mathbf{T}\mathbf{Q}^\top)}{\sigma_2^2}$.

Note that in (13), we set $\mathbf{P}$ and $\mathbf{Q}$ as orthogonal matrices, which follows the experimental settings of Li, Morris, and

Table 1. Comparison for different PLS algorithms.

Methods	Pre. cost[1–3]	Main cost	Method type	Acceleration type
PLS (Dayal and MacGregor 1997)	–	$\mathcal{O}(np^2 + mp^2l)$	–	–
UNIF	–	$\mathcal{O}(rp^2 + mp^2l)$	Randomized	Row Sampling
LEV	$\mathcal{O}(np^2)$	$\mathcal{O}(rp^2 + mp^2l)$	Randomized	Row Sampling
IFPLS (Xie, Feng, and Chen 2022)	$\mathcal{O}(np^2)$	$\mathcal{O}(rp^2 + mp^2l)$	Deterministic	Row Sampling
MCDPLS (Huang et al. 2024)	$\mathcal{O}(np^2L)$	$\mathcal{O}(rp^2 + mp^2l)$	Deterministic	Row Sampling
SGDPLS (Arora, Mianjy, and Marinov 2016)	–	$\mathcal{O}(mpL)$	Randomized	Batch Learning
CIPLS (Jordao et al. 2021)	–	$\mathcal{O}(npl)$	Randomized	Batch Learning
Spar-PLS (proposed)	$\mathcal{O}(\text{nnz}(\mathbf{X}))$	$\mathcal{O}(rp^2 + mp^2l)$	Randomized	Sparse Sampling

[1]Here, each method subsamples r rows or rp elements from the $n \times p$ predictor matrix.
[2]L denotes the number of iterations required for convergence in the respective methods.
[3]The computational complexities presented above are derived under the assumption that $l < p$.

Martin (2002); Lafaye de Micheaux, Liquet, and Sutton (2019). Specifically, Li, Morris, and Martin (2002) explored a low-dimensional scenario and proposed a specific set of orthogonal matrices, while Lafaye de Micheaux, Liquet, and Sutton (2019) ensured orthogonality by employing a simple sparse matrix. In this paper, we adopt a fixed set of orthogonal matrices for $\mathbf{P}$ and $\mathbf{Q}$ in the subsequent analysis to enhance the model's generalizability. In (14), we use Σ_1 to control the covariance structure of $\mathbf{T}$ and Σ_2 to control the heteroscedasticity of $\mathbf{T}$, which provides greater generalizability compared to Li, Morris, and Martin (2002); Lafaye de Micheaux, Liquet, and Sutton (2019).

For the experimental variables, we set the total sample size to $n = 50000$, the dimensionality of the observation matrix to $p = 100$, and the dimensionality of the response matrix to $l = 50$. We designate the latent variable dimensions as $m \in \{5, 10, 15, 20, 25\}$, denoted as (M1)-(M5). For the signal-to-noise ratios γ_1 and γ_2, we consider the following settings:

(R1) $\gamma_1 = \gamma_2 = 0.25$.
(R2) $\gamma_1 = \gamma_2 = 0.5$.
(R3) $\gamma_1 = \gamma_2 = 1$.

Finally, we evaluate subsample sizes of $r \in \{5p, 10p, 15p, 20p, 25p\}$, corresponding to matrix sparsities of {1%, 2%, 3%, 4%, 5%}.

Based on the settings above, we can derive the regression coefficient $\mathbf{B}$:

$$\mathbf{B} = \mathbf{P}\left(\mathbf{P}^\top\mathbf{P}\right)^{-1}\mathbf{Q}^\top. \tag{15}$$

Consequently, we can compute $\text{MSE}_\mathbf{B}$ using the estimated $\widehat{\mathbf{B}}$ obtained from different algorithms:

$$\text{MSE}_\mathbf{B} = \|\widehat{\mathbf{B}} - \mathbf{B}\|_F.$$

However, under this setting, $\mathbf{X}$ and $\mathbf{Y}$ are generated from the latent variable $\mathbf{T}$, as defined in (13). Consequently, the true $\mathbf{B}$ is given by

$$\mathcal{P}^\perp\mathbf{Q}^\top, \mathcal{P}^\perp = \{\mathbf{R} \in \mathbb{R}^{p\times m} \mid \mathbf{P}^\top\mathbf{R} = \mathbf{I}_m\}.$$

This implies that the true $\mathbf{B}$ in this scenario represents the projection of $\mathcal{P}^\perp$ onto the space spanned by $\mathbf{Q}^\top$. Therefore, directly employing $\text{MSE}_\mathbf{B}$ may not furnish a reliable measure of algorithmic performance, as it fails to account for the errors arising from variations within the column space of $\mathcal{P}^\perp$.

To address this issue, we consider eliminating the uncertainty of $\mathcal{P}^\perp$ by multiplying a $\mathbf{P}^\top$ in the left. Given that:

$$\mathbf{P}^\top\mathbf{B} = \mathbf{P}^\top\mathcal{P}^\perp\mathbf{Q}^\top = \mathbf{Q}^\top,$$

we are essentially comparing $\mathbf{P}^\top\widehat{\mathbf{B}}$ derived from the algorithm with the true value $\mathbf{Q}^\top$ and define the $\text{MSE}_\mathbf{Q}$ as:

$$\text{MSE}_\mathbf{Q} = \left\|\mathbf{P}^\top\widehat{\mathbf{B}} - \mathbf{Q}^\top\right\|_F. \tag{16}$$

Furthermore, we also compute the Prediction Mean Squared Error (PMSE) by partitioning the data into training and testing sets:

$$\text{PMSE} = \left\|\mathbf{X}_{\text{test}}\widehat{\mathbf{B}} - \mathbf{Y}_{\text{test}}\right\|_F,$$

where the parameter $\widehat{\mathbf{B}}$ is derived from the training set $\mathbf{X}_{\text{train}}, \mathbf{Y}_{\text{train}}$ using the various algorithms under consideration. Notably, we can decompose the PMSE as follows:

$$\left\|\mathbf{X}_{\text{test}}\widehat{\mathbf{B}} - \mathbf{Y}_{\text{test}}\right\|_F = \left\|\mathbf{T}_{\text{test}}(\mathbf{P}^\top\widehat{\mathbf{B}} - \mathbf{Q}^\top) + (\mathbf{E}\widehat{\mathbf{B}} - \mathbf{F})\right\|_F. \tag{17}$$

As the magnitude of this expression is primarily governed by $\mathbf{P}^\top\widehat{\mathbf{B}} - \mathbf{Q}^\top$, which is equivalent to the $\text{MSE}_\mathbf{Q}$ defined in (16), the PMSE also mitigates some uncertainty by using $\mathbf{P}^\top\widehat{\mathbf{B}}$.

By repeating the experiment 10 times, we obtained the results for $\text{MSE}_\mathbf{B}$ as depicted in Figure 3. This figure demonstrates the advantage of the sparsification subsampling method over the row sampling method. Additionally, the subsampling methods demonstrate robustness to variations in SNR and latent variable dimensions, with their performance progressively approaching that of the full-sample method as the sample size increases. However, a certain gap remains between the $\text{MSE}_\mathbf{B}$ achieved by sparsification subsampling and that of the full-sample method, potentially stemming from the inherent uncertainty in the estimation of $\mathbf{B}$.

Meanwhile, Figure 4 presents the results for $\text{MSE}_\mathbf{Q}$, illustrating the significant advantage of the sparsification subsampling method over the row sampling method in this context. Notably, with a sample size of 1000 (corresponding to a 2% sampling ratio), the $\text{MSE}_\mathbf{Q}$ achieved by sparsification subsampling closely approximates that of the full-sample method. This finding suggests that the sparsification subsampling method demonstrates superior predictive performance in exploring the direct relationship between latent variables and their corresponding variables.

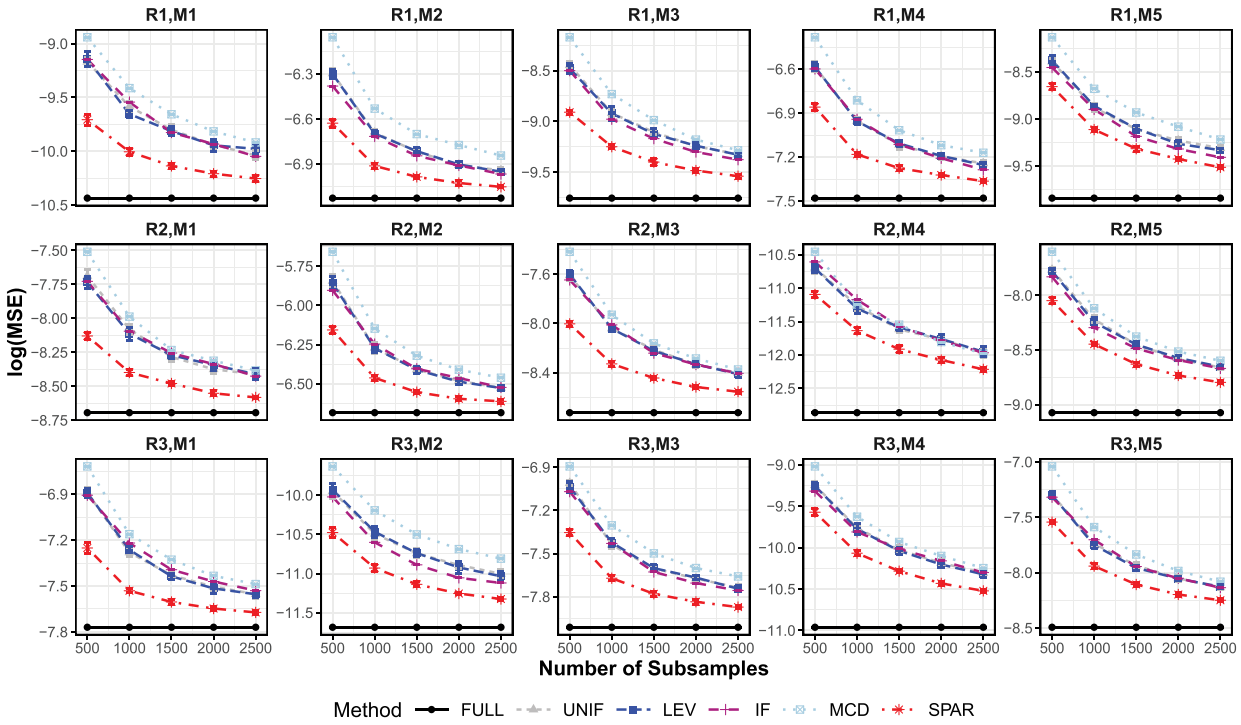


Figure 3. Comparison of MSE_B for different algorithms across varying numbers of subsamples r. each row represents the different SNR settings, that is (R1-R3), and each column represents the different latent variable dimensions, that is (M1-M5).

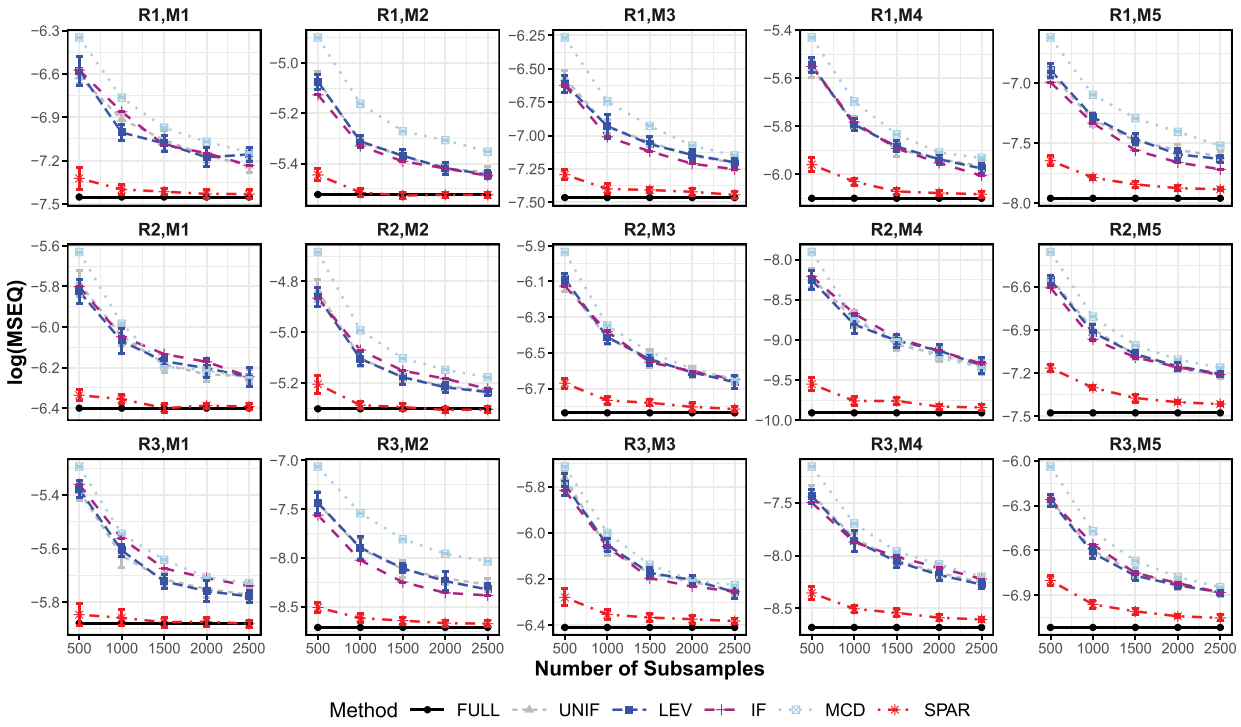


Figure 4. Comparison of MSE_Q for different algorithms across varying numbers of subsamples r. each row represents the different SNR settings, that is (R1-R3), and each column represents the different latent variable dimensions, that is (M1-M5).

The PMSE results obtained are presented in Figure 5, which further corroborates the superiority of the sparsification subsampling method, demonstrating consistently lower PMSE values compared to the row sampling method. It is worth noting that the variability is primarily driven by cross-validation; notably, subsampling methods exhibit variance comparable to the full-sample approach. Interestingly, the PMSE values achieved by sparsification subsampling lie between those of MSE_B and MSE_Q. This observation aligns with the decomposition of PMSE derived in (17), suggesting that PMSE reflects the model's performance after partially mitigating the uncertainty associated with the estimation of $\mathbf{B}$.

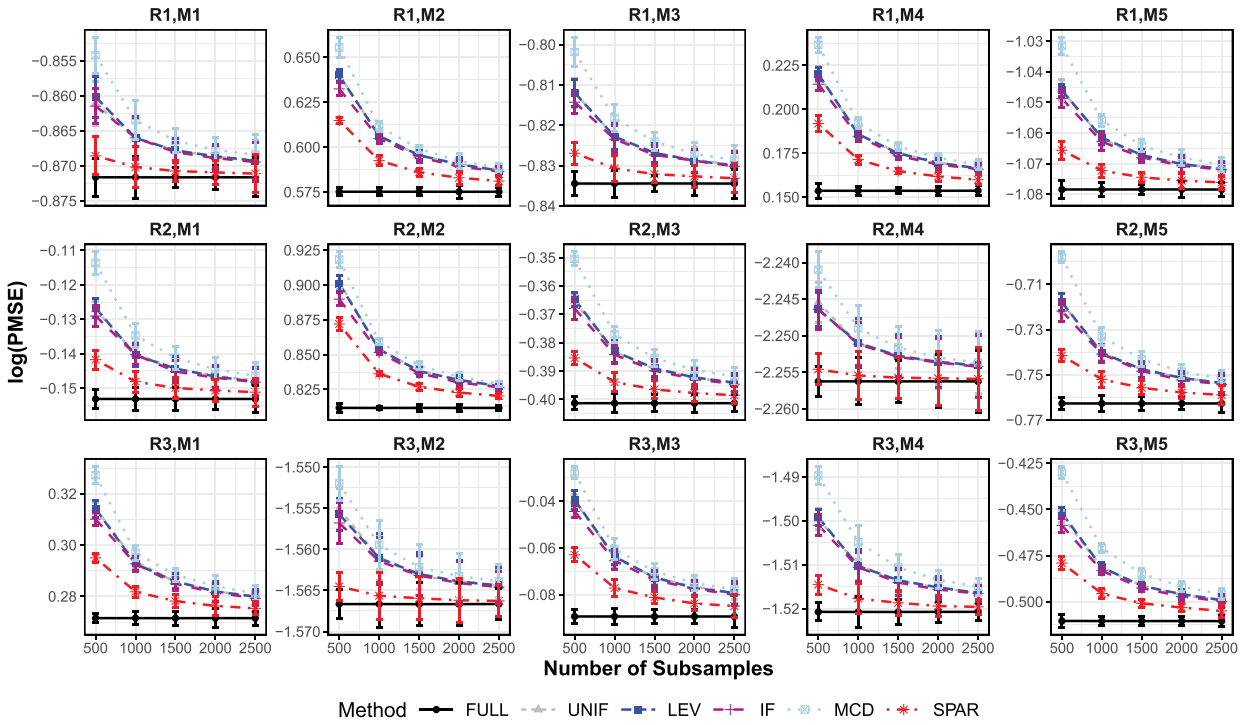


Figure 5. Comparison of PMSE for different algorithms across varying numbers of subsamples *r*. each row represents the different SNR settings, that is (R1-R3), and each column represents the different latent variable dimensions, that is (M1-M5).

Table 2. Runtime comparison of different PLS implementations (s).

# Components	m = 5					m = 10				
# Subsamples	500	1000	1500	2000	2500	500	1000	1500	2000	2500
FULL	0.447	0.465	0.456	0.479	0.460	0.682	0.684	0.643	0.651	0.659
UNIF	0.006	0.009	0.015	0.017	0.022	0.009	0.016	0.027	0.046	0.031
LEV	0.689	0.712	0.706	0.706	0.728	0.701	0.720	0.705	0.707	0.709
IF	1.253	1.252	1.248	1.265	1.278	1.497	1.473	1.511	1.483	1.459
MCD	1.085	1.078	1.084	1.073	1.093	1.067	1.097	1.090	1.089	1.093
SPAR	0.030	0.036	0.049	0.046	0.051	0.035	0.042	0.050	0.056	0.063

To intuitively demonstrate the efficiency of the sparse algorithm, we compare the running times of various algorithms, including the probability computation, subsampling, and main algorithm execution phases. We set $n = 50000$, $p = 100$, and $l = 50$. The number of principal components, m, varies between 5 and 10, while the number of sampled elements, r, ranges from 500 to 2500 in increments of 500. We repeat the experiment 10 times and compare the total times of all competing methods. The results are presented in Table 2.

Table 2 compares the runtime for different algorithms. The results indicate that, with sampling ratios ranging from 1% to 5%, the sparsification method achieves a consistent speedup of approximately 10 times compared to the full-sample method. While the runtime of the sparsification method is slightly longer than that of the row sampling method, primarily due to the overhead associated with sparse matrix operations, this difference is expected to diminish as the data scale increases. Furthermore, the runtime of the Spar-PLS method on large-scale datasets exhibits low sensitivity to variations in the number of iterations (m), further validating the effectiveness of the subsampling methods in accelerating computations.

5. Real Data Analysis

Wave energy, a rapidly developing and highly promising renewable energy source, holds substantial potential for addressing the challenges of global warming and climate change. However, optimizing the energy output of large-scale wave farms is a complex task, requiring computationally expensive simulations to account for the hydrodynamic interactions between wave energy converters (WECs). The development of fast and accurate surrogate models is crucial to overcoming these computational challenges. To this end, Neshat et al. (2020) compiled a comprehensive dataset of WEC arrays, consisting of real-world measurements collected from wave energy converters deployed in offshore scenarios (Perth and Sydney) to detect and calculate energy generation at specific locations. In each experimental setup, either 49 or 100 detectors were positioned at various locations. Consequently, the predictor variables consist of the 98-dimensional or 200-dimensional coordinate information (representing the 2D coordinates of the detectors), while the response variables correspond to the generated power output, having dimensions of 49 or 100, respectively.

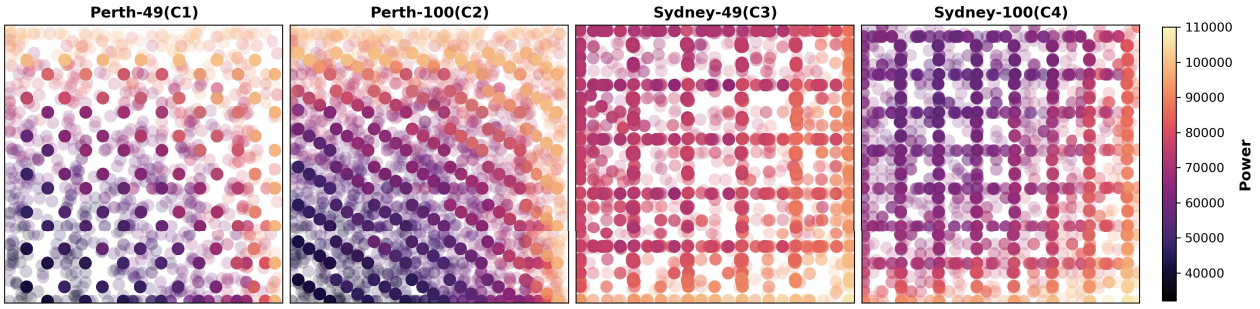


Figure 6. Illustration of four scenarios from the WEC dataset. Each point represents the physical coordinates of a WEC, with the color intensity indicating the calculated power output at that location.

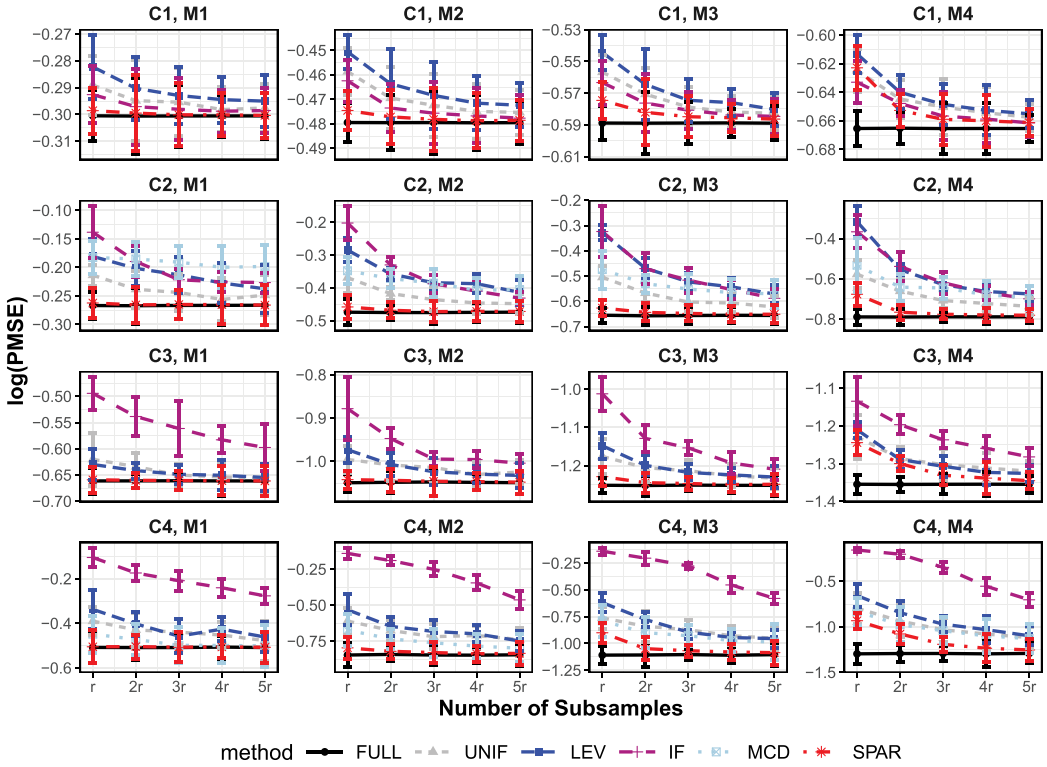


Figure 7. Comparison of PMSE for different algorithms across varying numbers of subsamples r. each row represents the different scenarios, that is (C1-C4), and each column represents the different latent variable dimensions, that is (M1-M4).

As illustrated in Figure 6, the aggregated data from numerous trials reveals a clear gradient pattern in the energy field relative to the physical positions. This spatial dependence motivates the application of the PLS method to model the relationship between the detector locations and the corresponding power outputs.

For the four scenarios, encompassing 49 and 100 WECs combined with wave conditions from Perth and Sydney (denoted as (C1)-(C4)), we select the number of iterations $m \in \{5, 10, 15, 20\}$ (denoted as (M1)-(M4)). For each scenario, we employ 10-fold cross-validation, setting the subsample size to 2%, 4%, 6%, 8%, 10% of the total sample size. The mean and variance of the Prediction Mean Squared Error are computed for each fold and presented in Figure 7.

As illustrated in Figure 7, the sparsification subsampling method consistently outperforms row subsampling once the sample size stabilizes relative to the number of iterations. This observation aligns with the findings from the simulation study.

Table 3. Comparison of running times on real data (s).

#Components	m = 5	m = 10	m = 15	m = 20
FULL	0.330	0.480	0.622	0.777
UNIF	0.007	0.009	0.012	0.016
LEV	0.377	0.383	0.384	0.384
IF	0.848	0.992	1.138	1.363
MCD	0.491	0.490	0.493	0.544
SPAR	0.042	0.044	0.045	0.066

Furthermore, we compare the runtime of different algorithms. For this comparison, we utilize the Perth49 dataset, which has the largest sample size, with the subsample size set to $r = 500$ and the number of iterations $m \in \{5, 10, 15, 20\}$. The results are presented below.

As shown in Table 3, the sparse sampling algorithm still achieves a speedup of 80% − 90%, consistent with the conclusions from the previous numerical simulations.

6. Conclusion

Partial Least Squares is an effective supervised dimensionality reduction method. To address the high computational complexity of PLS on large-scale data, we propose a novel Spar-PLS algorithm that applies sparse subsampling techniques to PLS regression. By theoretically setting the subsampling probabilities and applying matrix sparsification approximations, Spar-PLS efficiently handles data with both large-scale and high-dimensional characteristics. Numerical simulations and real data analyses demonstrate that, compared to the conventional PLS algorithm and other sampling strategies, the proposed Spar-PLS method achieves superior performance with enhanced computational efficiency.

Future research directions for the Spar-PLS method are manifold. On the one hand, many studies suggest that the PLS regression coefficient is a biased estimate (Phatak, Reilly, and Penlidis 2002), thus finding a covariance approximation aims to obtain a regression coefficient with less bias, achieving local optimality. While this study derives the minimum variance approximation for matrix multiplication under the Frobenius norm, further investigation for approximations under the spectral norm and other metrics is warranted. On the other hand, the theoretical properties of PLS are currently only for the single-component and univariate response cases (Nadler and Coifman 2005; Cook and Forzani 2019). Future work may derive subsampling probabilities based on existing theoretical properties of the parameters, or by simulating the parameter variance (Odgers et al. 2023) and set probabilities by parameter variance like several classical methods (Wang, Zhu, and Ma 2018; Han et al. 2025). In conclusion, further research is needed to explore the asymptotic properties of PLS regression, to determine the optimal sampling probabilities for sparsification algorithms under different norms, and to analyze the convergence behavior during the algorithm's iterative process.

Supplementary Material

Appendix:
Provides complete proofs of the theoretical results. (appendix.pdf, a pdf file)
Code:
Includes R code for implementing the proposed method and reproducing the numerical results. (code.zip, a zip file)

Disclosure Statement

No potential conflict of interest was reported by the author(s).

Funding

This research was funded by the National Natural Science Foundation of China (No. 12131001), the Fundamental Research Funds for the Central Universities (No. 63263100), and the China Postdoctoral Science Foundation under Grant Number 2025M783111.

ORCID

Junyi Lin http://orcid.org/0009-0009-1715-2725
Mengyu Li http://orcid.org/0000-0002-5286-7525

References

Achlioptas, D., and Mcsherry, F. (2007), "Fast Computation of Low-Rank Matrix Approximations," *Journal of the ACM*, 54, 9-es. DOI: 10.1145/1219092.1219097. [2]

Ai, M., Wang, F., Yu, J., and Zhang, H. (2021a), "Optimal Subsampling for Large-Scale Quantile Regression," *Journal of Complexity*, 62, 101512. DOI: 10.1016/j.jco.2020.101512. [2]

Ai, M., Yu, J., Zhang, H., and Wang, H. (2021b), "Optimal Subsampling Algorithms for Big Data Regressions," *Statistica Sinica*, 31, 749–772. DOI: 10.5705/ss.202018.0439. [2]

Arora, R., Mianjy, P., and Marinov, T. (2016), "Stochastic Optimization for Multiview Representation Learning Using Partial Least Squares," in *Proceedings of The 33rd International Conference on Machine Learning*, Vol. 48, pp. 1786–1794, New York, NY: PMLR. https://proceedings.mlr.press/v48/aroraa16.html. [1,10]

Braverman, V., Krauthgamer, R., Krishnan, A. R., and Sapir, S. (2021), "Near-Optimal Entrywise Sampling of Numerically Sparse Matrices," in *Proceedings of Thirty Fourth Conference on Learning Theory*, Vol. 134, pp. 759–773, PMLR. https://proceedings.mlr.press/v134/braverman21b.html. [5]

Cook, R. D., and Forzani, L. (2019), "Partial Least Squares Prediction in High-Dimensional Regression," *The Annals of Statistics*, 47, 884–908. DOI: 10.1214/18-AOS1681. [14]

Cook, R. D., and Forzani, L.. (2024), *Partial Least Squares Regression: And Related Dimension Reduction Methods*, New York: Chapman and Hall/CRC. 10.1201/9781003482475. [1]

Cook, R. D., Helland, I. S., and Su, Z. (2013), "Envelopes and Partial Least Squares Regression," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 75, 851–877. DOI: 10.1111/rssb.12018. [3]

Dayal, B. S., and MacGregor, J. F. (1997), "Improved PLS Algorithms," *Journal of Chemometrics*, 11, 73–85. DOI: 10.1002/(SICI)1099-128X(199701)11:1<73::AID-CEM435>3.0.CO;2-#. [1,2,3,8,10]

De Jong, S. (1993), "SIMPLS: An Alternative Approach to Partial Least Squares Regression," *Chemometrics and Intelligent Laboratory Systems*, 18, 251–263. DOI: 10.1016/0169-7439(93)85002-X. [1,3]

Drineas, P., Kannan, R., and Mahoney, M. W. (2006), "Fast Monte Carlo Algorithms for Matrices I: Approximating Matrix Multiplication," *SIAM Journal on Computing*, 36, 132–157. DOI: 10.1137/S0097539704442684. [3,5,6]

Drineas, P., Mahoney, M. W., and Muthukrishnan, S. (2006). "Sampling Algorithms for l_2 Regression and Applications," in *Proceedings of the seventeenth annual ACM-SIAM symposium on Discrete algorithm*, pp. 1127–1136. USA: Society for Industrial and Applied Mathematics. DOI: 10.1145/1109557.1109682. [1,2]

Feng, Y., and Zhou, Y. (2024), "GGD: Grafting Gradient Descent," *Journal of Machine Learning Research*, 25, 1–87. [2]

Han, Y., Yu, J., Zhang, N., Meng, C., Ma, P., Zhong, W., and Zou, C. (2025), "Leverage Classifier: Another Look at Support Vector Machine," *Statistica Sinica*, 35, 1605–1625. DOI: 10.5705/ss.202023.0124. [2,14]

Höskuldsson, A. (1988), "PLS Regression Methods," *Journal of Chemometrics*, 2, 211–228. DOI: 10.1002/cem.1180020306. [1,3]

Hu, Y., Li, M., Liu, X., and Meng, C. (2024), "Sampling-Based Methods for Multi-Block Optimization Problems over Transport Polytopes," *Mathematics of Computation*, 94, 1281–1322. DOI: 10.1090/mcom/3989. [2]

Huang, J., Kang, X., Huang, Q., Li, M., Meng, C., and Zhang, J. (2026), "Efficient Approximation of Leverage Scores in Two-Dimensional Autoregressive Models with Application to Image Anomaly Detection," *Journal of Computational and Graphical Statistics*, 35, 89–100. DOI: 10.1080/10618600.2025.2505732. [2]

Huang, X., Huang, G., Chen, X., Xie, Z., Ali, S., Chen, X., Yuan, L., and Shi, W. (2024), "An Adaptive Strategy to Improve the Partial Least Squares Model via Minimum Covariance Determinant," *Chemometrics and Intelligent Laboratory Systems*, 249, 105120. DOI: 10.1016/j.chemolab.2024.105120. [2,9,10]

Jordao, A., Lie, M., De Melo, V. H. C., and Schwartz, W. R. (2021), "Covariance-Free Partial Least Squares: An Incremental Dimensionality Reduction Method," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 1421–1429. https://ieeexplore.ieee.org/abstract/document/9423374. [1,10]

Kelly, B., and Pruitt, S. (2015), "The Three-Pass Regression Filter: A New Approach to Forecasting Using Many Predictors," *Journal of Econometrics*, 186, 294–316. DOI: 10.1016/j.jeconom.2015.02.011. [3]

Kundu, A., Drineas, P., and Magdon-Ismail, M. (2017), "Recovering PCA and Sparse PCA via Hybrid-(l1,l2) Sparse Sampling of Data Elements," *Journal of Machine Learning Research*, 18, 1–34. [2]

Lafaye de Micheaux, P., Liquet, B., and Sutton, M. (2019), "PLS for Big Data: A Unified Parallel Algorithm for Regularised Group PLS," *Statistics Surveys*, 13, 119–149. DOI: 10.1214/19-SS125. [1,10]

Li, B., Morris, J., and Martin, E. B. (2002), "Model Selection for Partial Least Squares Regression," *Chemometrics and Intelligent Laboratory Systems*, 64, 79–89. DOI: 10.1016/S0169-7439(02)00051-5. [10]

Li, M., Yu, J., Li, T., and Meng, C. (2023a), "Importance Sparsification for Sinkhorn Algorithm," *Journal of Machine Learning Research*, 24, 1–44. [2]

Li, M., Yu, J., Li, T., and Meng, C.. (2024), "Core-Elements for Large-Scale Least Squares Estimation," *Statistics and Computing*, 34, 1–16. DOI: 10.1007/s11222-024-10505-6. [2,5]

Li, M., Yu, J., Xu, H., and Meng, C. (2023b), "Efficient Approximation of Gromov-Wasserstein Distance Using Importance Sparsification," *Journal of Computational and Graphical Statistics*, 32, 1512–1523. DOI: 10.1080/10618600.2023.2165500. [2]

Li, M., Zhang, J., and Meng, C. (2024), "Nonparametric Additive Models for Billion Observations," *Journal of Computational and Graphical Statistics*, 33, 1397–1412. DOI: 10.1080/10618600.2024.2319684. [2]

Li, T., and Meng, C. (2020), "Modern Subsampling Methods for Large-Scale Least Squares Regression," *International Journal of Cyber-Physical Systems*, 2, 1–28. DOI: 10.4018/IJCPS.2020070101. [2]

Lindgren, F., Geladi, P., and Wold, S. (1993), "The Kernel Algorithm for PLS," *Journal of Chemometrics*, 7, 45–59. DOI: 10.1002/cem.1180070104. [1,2,3]

Ma, P., Chen, Y., Zhang, X., Xing, X., Ma, J., and Mahoney, M. W. (2022), "Asymptotic Analysis of Sampling Estimators for Randomized Numerical Linear Algebra Algorithms," *Journal of Machine Learning Research*, 23, 1–45. [1,2]

Ma, P., Huang, J. Z., and Zhang, N. (2015), "Efficient Computation of Smoothing Splines via Adaptive Basis Sampling," *Biometrika*, 102, 631–645. DOI: 10.1093/biomet/asv009. [2]

Ma, P., Mahoney, M., and Yu, B. (2014), "A Statistical Perspective on Algorithmic Leveraging," In *Proceedings of the 31st International Conference on Machine Learning*, Bejing, China, Vol. 32, pp. 91–99, PMLR. https://proceedings.mlr.press/v32/ma14.html. [1,2]

Ma, P., and Sun, X. (2015), "Leveraging for Big Data Regression," *WIREs Computational Statistics*, 7, 70–76. DOI: 10.1002/wics.1324. [1,2]

Mahoney, M. W. (2016), "Lecture Notes on Randomized Linear Algebra," *arXiv preprint*, arXiv:1608.04481. https://arxiv.org/abs/1608.04481. [4,5]

Meng, C., Wang, Y., Zhang, X., Mandal, A., Zhong, W., and Ma, P. (2017), "Effective Statistical Methods for Big Data Analytics," in *Handbook of Research on Applied Cybernetics and Systems Science*, pp. 280–299, IGI Global Scientific Publishing. DOI: 10.4018/978-1-5225-2498-4.ch014. [2]

Meng, C., Xie, R., Mandal, A., Zhang, X., Zhong, W., and Ma, P. (2021), "Lowcon: A Design-Based Subsampling Approach in a Misspecified Linear Model," *Journal of Computational and Graphical Statistics*, 30, 694–708. DOI: 10.1080/10618600.2020.1844215. [1,2]

Meng, C., Yu, J., Chen, Y., Zhong, W., and Ma, P. (2022), "Smoothing Splines Approximation Using Hilbert Curve Basis Selection," *Journal of Computational and Graphical Statistics: a Joint Publication of American Statistical Association, Institute of Mathematical Statistics, Interface Foundation of North America*, 31, 802–812. DOI: 10.1080/10618600.2021.2002161. [2]

Meng, C., Zhang, X., Zhang, J., Zhong, W., and Ma, P. (2020), "More Efficient Approximation of Smoothing Splines via Space-Filling Basis Selection," *Biometrika*, 107, 723–735. DOI: 10.1093/biomet/asaa019. [2]

Nadler, B., and Coifman, R. R. (2005), "Partial Least Squares, Beer's Law and the Net Analyte Signal: statistical Modeling and Analysis," *Journal of Chemometrics*, 19, 45–54. DOI: 10.1002/cem.906. [14]

Neshat, M., Alexander, B., Sergiienko, N. Y., and Wagner, M. (2020), "Optimisation of Large Wave Farms Using a Multi-Strategy Evolutionary Framework," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, pp. 1150–1158, New York, NY: Association for Computing Machinery. [12]

Odgers, J., Kappatou, C., Misener, R., García Muñoz, S., and Filippi, S. (2023), "Probabilistic Predictions for Partial Least Squares Using Bootstrap," *AIChE Journal*, 69, e18071. DOI: 10.1002/aic.18071. [14]

Ouyang, X., Zheng, H., Liang, H., Zhang, J., Qiu, Y., Meng, C., and Li, M. (2026), "Sparsification Techniques for Large-Scale Optimal Transport Problems," *Wiley Interdisciplinary Reviews: Computational Statistics*, 18, e70056. [2]

Phatak, A., Reilly, P., and Penlidis, A. (2002), "The Asymptotic Variance of the Univariate Pls Estimator," *Linear Algebra and Its Applications*, 354, 245–253. DOI: 10.1016/S0024-3795(01)00357-3. [14]

Schwartz, W. R., Guo, H., and Davis, L. S. (2010), "A Robust and Scalable Approach to Face Identification," in *Computer Vision–ECCV 2010: 11th European Conference on Computer Vision, Heraklion, Crete, Greece, September 5-11, 2010, Proceedings, Part VI 11*, pp. 476–489. Berlin, Heidelberg: Springer. [1]

Stocchero, M. (2019), "Iterative Deflation Algorithm, Eigenvalue Equations, and PLS2," *Journal of Chemometrics*, 33, e3144. DOI: 10.1002/cem.3144. [3]

Stott, A. E., Kanna, S., Mandic, D. P., and Pike, W. T. (2017), "An Online NIPALS Algorithm for Partial Least Squares," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4177–4181. New Orleans, LA: IEEE. DOI: 10.1109/ICASSP.2017.7952943. [1]

Tabei, Y., Saigo, H., Yamanishi, Y., and Puglisi, S. J. (2016), "Scalable Partial Least Squares Regression on Grammar-Compressed Data Matrices," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1875–1884, New York, NY: Association for Computing Machinery. [1]

Ting, D., and Brochu, E. (2018), "Optimal Subsampling with Influence Functions," in *Advances in Neural Information Processing Systems*, Volume 31, Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/57c0531e13f40b91b3b0f1a30b529a1d-Paper.pdf. [1,2]

Trygg, J., and Wold, S. (2002), "Orthogonal Projections to Latent Structures (O-PLS)," *Journal of Chemometrics*, 16, 119–128. DOI: 10.1002/cem.695. [3]

Wang, H., and Ma, Y. (2021), "Optimal Subsampling for Quantile Regression in Big Data," *Biometrika*, 108, 99–112. DOI: 10.1093/biomet/asaa043. [2]

Wang, H., Yang, M., and Stufken, J. (2019), "Information-Based Optimal Subdata Selection for Big Data Linear Regression," *Journal of the American Statistical Association*, 114, 393–405. DOI: 10.1080/01621459.2017.1408468. [1,2]

Wang, H., Zhu, R., and Ma, P. (2018), "Optimal Subsampling for Large Sample Logistic Regression," *Journal of the American Statistical Association*, 113, 829–844. DOI: 10.1080/01621459.2017.1292914. [2,14]

Wang, L., Elmstedt, J., Wong, W. K., and Xu, H. (2021), "Orthogonal Subsampling for Big Data Linear Regression," *The Annals of Applied Statistics*, 15, 1273–1290. DOI: 10.1214/21-AOAS1462. [1,2]

Wang, Y., Yu, A. W., and Singh, A. (2017), "On Computationally Tractable Selection of Experiments in Measurement-Constrained Regression Models," *Journal of Machine Learning Research*, 18, 1–41. [2]

Wang, Y.-X., Balle, B., and Kasiviswanathan, S. P. (2019), "Subsampled Rényi Differential Privacy and Analytical Moments Accountant," in *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, Vol. 89, pp. 1226–1235, PMLR. https://proceedings.mlr.press/v89/wang19b.html. [2]

Wold, H. (1966), "Estimation of Principal Components and Related Models by Iterative Least Squares," *Multivariate Analysis*, 391–420. [1,3]

Wold, S., Ruhe, A., Wold, H., and Dunn, Iii, W. (1984), "The Collinearity Problem in Linear Regression. the Partial Least Squares (PLS) Approach to Generalized Inverses," *SIAM Journal on Scientific and Statistical Computing*, 5, 735–743. DOI: 10.1137/0905052. [1]

Wu, X., Huo, Y., Ren, H., and Zou, C. (2024), "Optimal Subsampling via Predictive Inference," *Journal of the American Statistical Association*, 119, 2844–2856. DOI: 10.1080/01621459.2023.2282644. [1]

Xiao, H., Rasul, K., and Vollgraf, R. (2017), "Fashion-Mnist: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint,* arXiv:1708.07747. https://arxiv.org/abs/1708.07747. [2]

Xie, R., Bai, S., and Ma, P. (2023), "Optimal Sampling Designs for Multidimensional Streaming Time Series with Application to Power Grid Sensor Data," *The Annals of Applied Statistics*, 17, 3195–3215. DOI: 10.1214/23-AOAS1757. [2]

Xie, R., Wang, Z., Bai, S., Ma, P., and Zhong, W. (2019), "Online Decentralized Leverage Score Sampling for Streaming Multidimensional Time Series," in *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, Vol. 89, pp. 2301—-2311, PMLR. https://proceedings.mlr.press/v89/xie19a.html. [2]

Xie, Z., Feng, X., and Chen, X.,. (2022), "Subsampling for Partial Least-Squares Regression via an Influence Function," *Knowledge-Based Systems*, 245, 108661. DOI: 10.1016/j.knosys.2022.108661. [2,9,10]

Xue, D., Li, M., Zhang, J., and Meng, C. (2026), "Core-Elements Subsampling for Alternating Least Squares," *Journal of Computational and Graphical Statistics*, 1–26. DOI: 10.1080/10618600.2026.2653762. [2]

Yi, S.-Y., and Zhou, Y.-D. (2023), "Model-Free Global Likelihood Subsampling for Massive Data," *Statistics and Computing*, 33, 9. DOI: 10.1007/s11222-022-10185-0. [1,2]

Yu, J., Ai, M., and Ye, Z. (2024), "A Review on Design Inspired Subsampling for Big Data," *Statistical Papers*, 65, 467–510. DOI: 10.1007/s00362-022-01386-w. [2]

Yu, J., Liu, J., and Wang, H. (2023), "Information-Based Optimal Subdata Selection for Non-Linear Models," *Statistical Papers*, 64, 1069–1093. DOI: 10.1007/s00362-023-01430-3. [1]

Yu, J., Wang, H., Ai, M., and Zhang, H. (2022), "Optimal Distributed Subsampling for Maximum Quasi-Likelihood Estimators with Massive Data," *Journal of the American Statistical Association*, 117, 265–276. DOI: 10.1080/01621459.2020.1773832. [2]

Yu, J., Wang, H., and Ai, M. (2025), "A Subsampling Strategy for AIC-Based Model Averaging with Generalized Linear Models," *Technometrics*, 67, 122–132. DOI: 10.1080/00401706.2024.2407310. [2]

Zeng, X.-Q., and Li, G.-Z. (2014), "Incremental Partial Least Squares Analysis of Big Streaming Data," *Pattern Recognition*, 47, 3726–3735. DOI: 10.1016/j.patcog.2014.05.022. [1]

Zhang, M., Zhou, Y., Zhou, Z., and Zhang, A. (2024a), "Model-Free Subsampling Method Based on Uniform Designs," *IEEE Transactions on Knowledge and Data Engineering*, 36, 1210–1220. DOI: 10.1109/TKDE.2023.3297167. [1,2]

Zhang, Y., Wang, L., Zhang, X., and Wang, H. (2024b), "Independence-Encouraging Subsampling for Nonparametric Additive Models," *Journal of Computational and Graphical Statistics*, 33, 1424–1433. DOI: 10.1080/10618600.2024.2326136. [2]

Zhou, Z., Yang, Z., Zhang, A., and Zhou, Y. (2024), "Efficient Model-Free Subsampling Method for Massive Data," *Technometrics*, 66, 240–252. DOI: 10.1080/00401706.2023.2271091. [1,2]